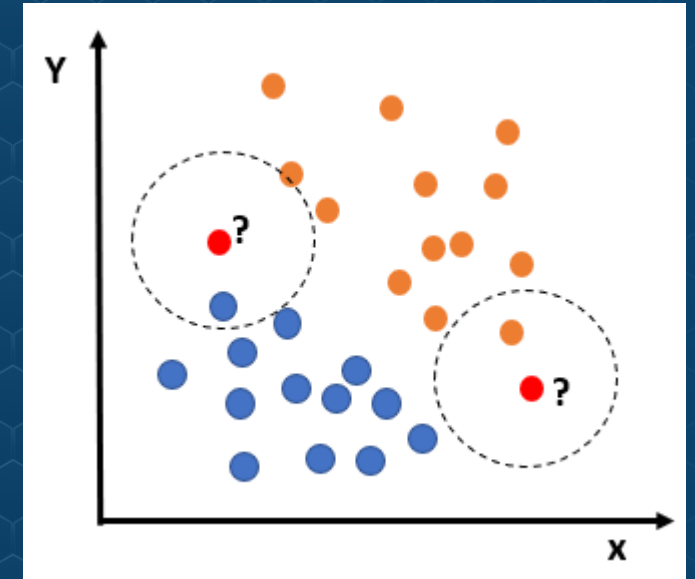




Phân loại ảnh với KNN (Image Classification with KNN)



Thuật toán KNN





Giới thiệu

- Là một thuật toán máy học áp dụng cho **bài toán phân loại**
- Thuộc nhóm thuật toán **Lazy Learning**: không học/ xây dựng mô hình mà lưu trữ toàn bộ dữ liệu huấn luyện

Mô tả thuật toán

Input:

- $X_{\text{train}}, y_{\text{train}}$: Tập dữ liệu huấn luyện X_{train} đã được gán nhãn y_{train}
- x : điểm dữ liệu cần phân loại
- **Số lượng láng giềng** ($n_{\text{neighbors}}$): K

Output:

- $\hat{y}$: Nhãn của điểm dữ liệu x

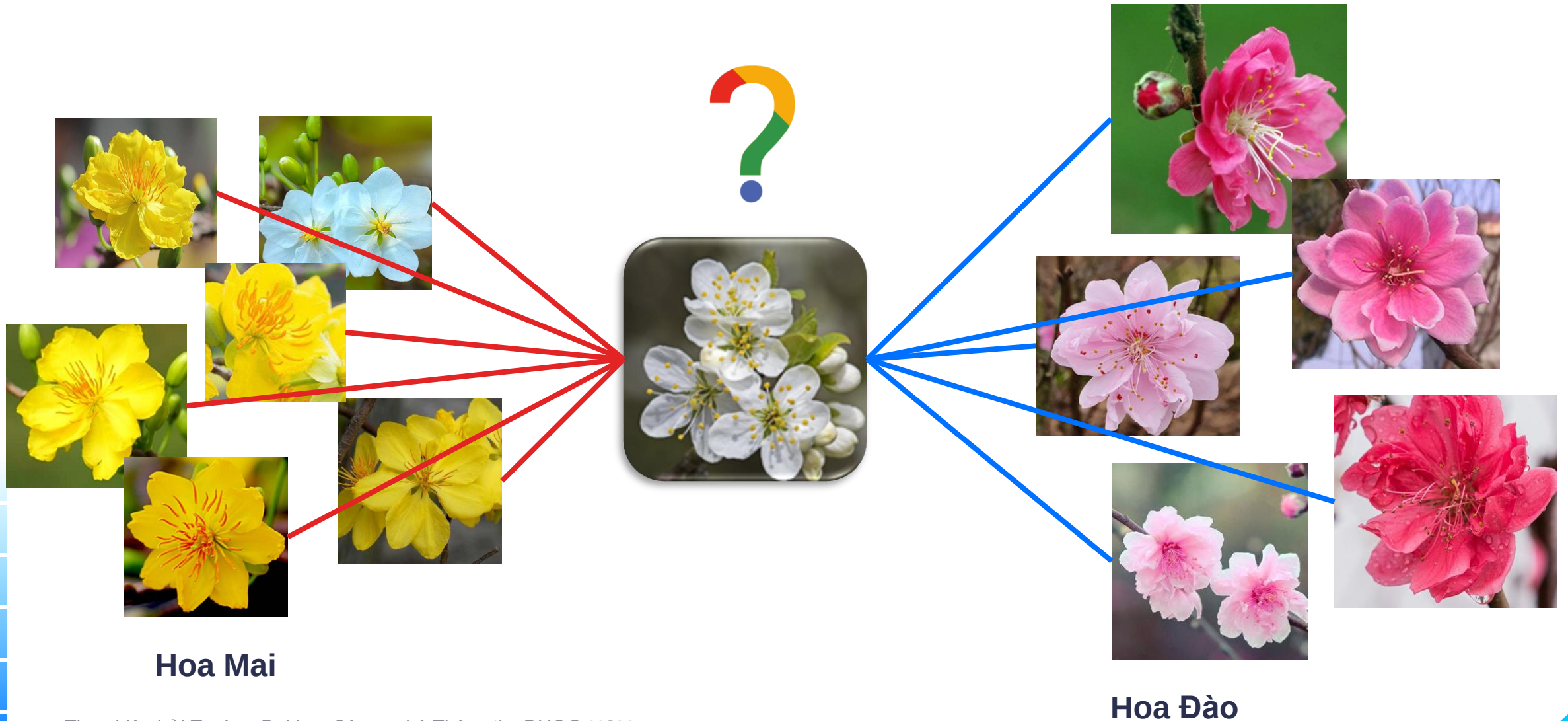
Bước 1:

Tính khoảng cách giữa điểm cần phân loại và tất cả các điểm trong tập huấn luyện.

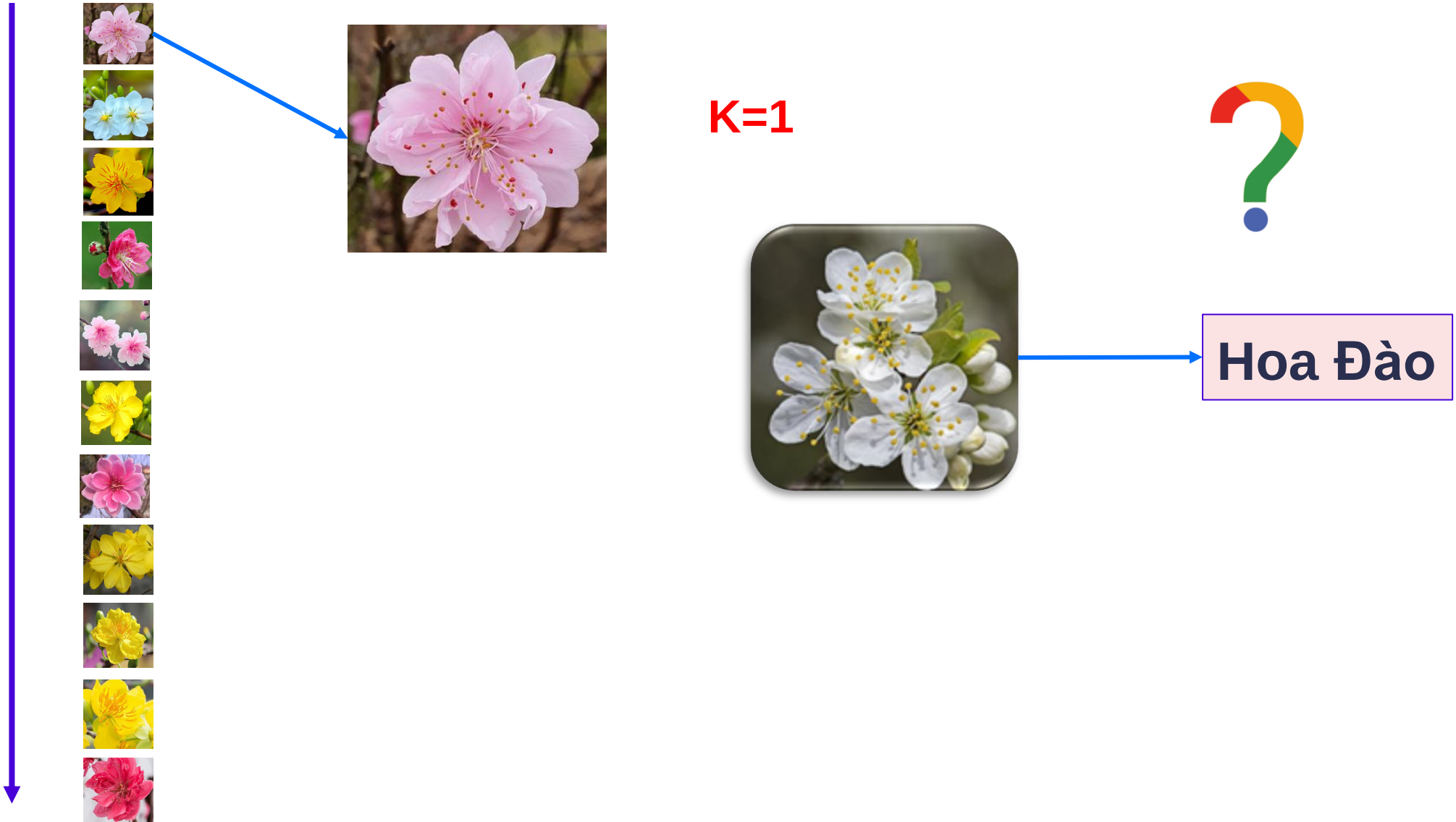
Bước 2:

Chọn K điểm gần nhất để **xác định kết quả**.

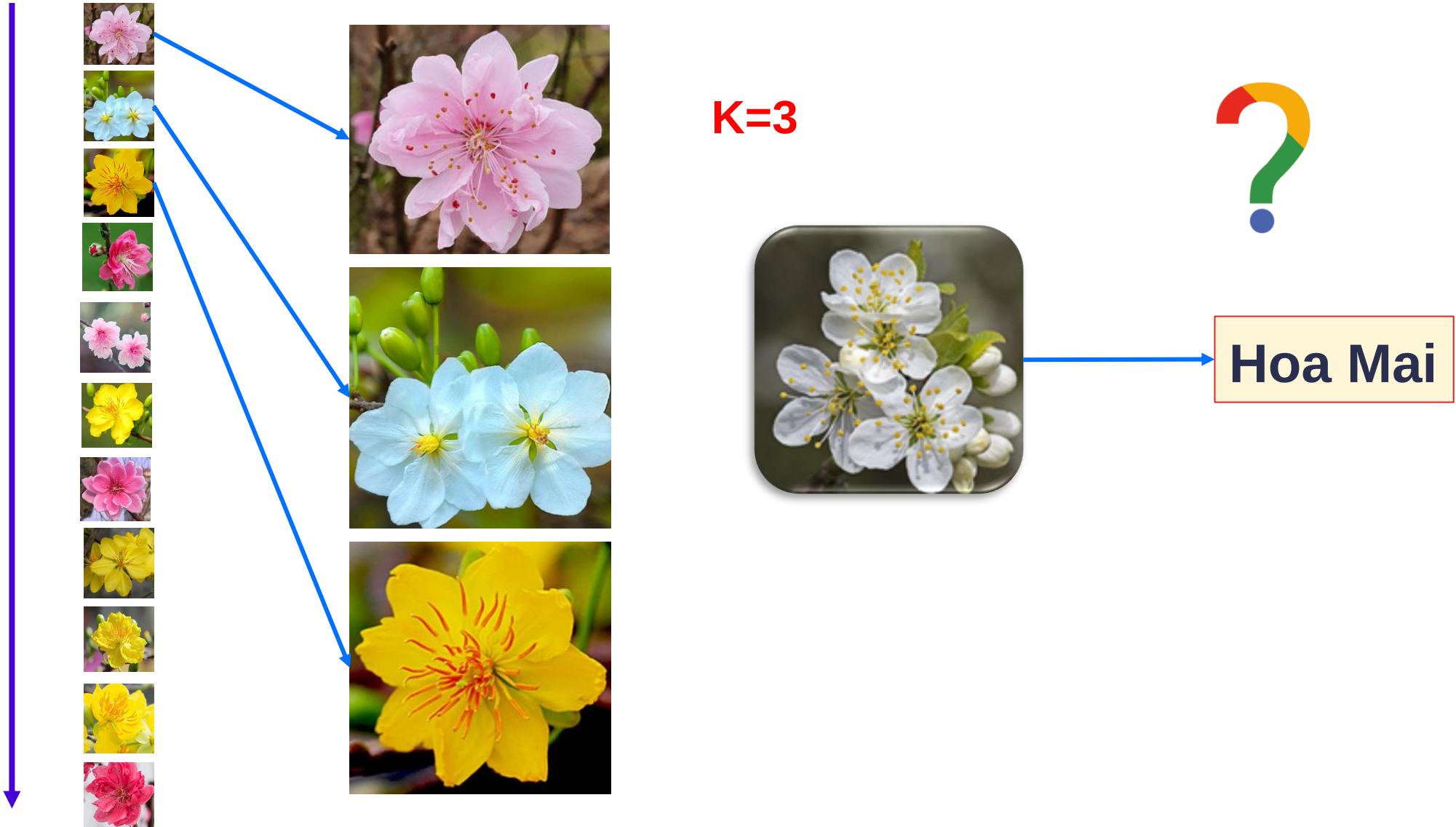
Ví dụ



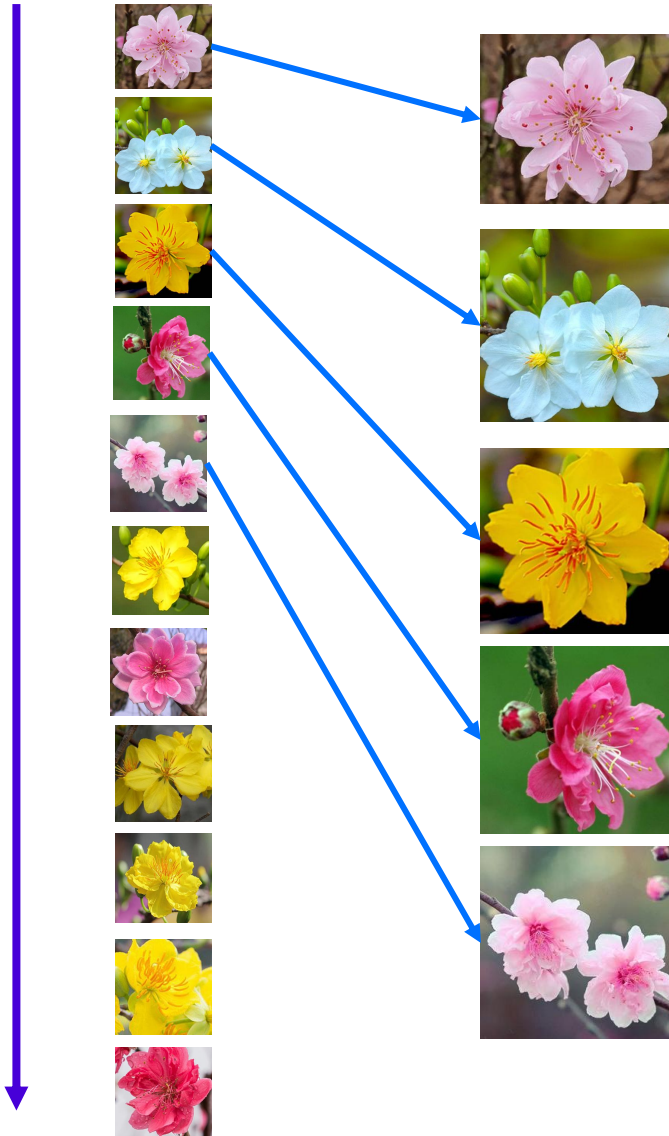
Ví dụ



Ví dụ



Ví dụ



K=5



Hoa Đào

Độ đo khoảng cách

- Có nhiều độ đo khoảng cách
- Phổ biến:

Distance functions

Euclidean

$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

Manhattan

$$\sum_{i=1}^k |x_i - y_i|$$

Minkowski

$$\left(\sum_{i=1}^k (|x_i - y_i|)^q \right)^{1/q}$$

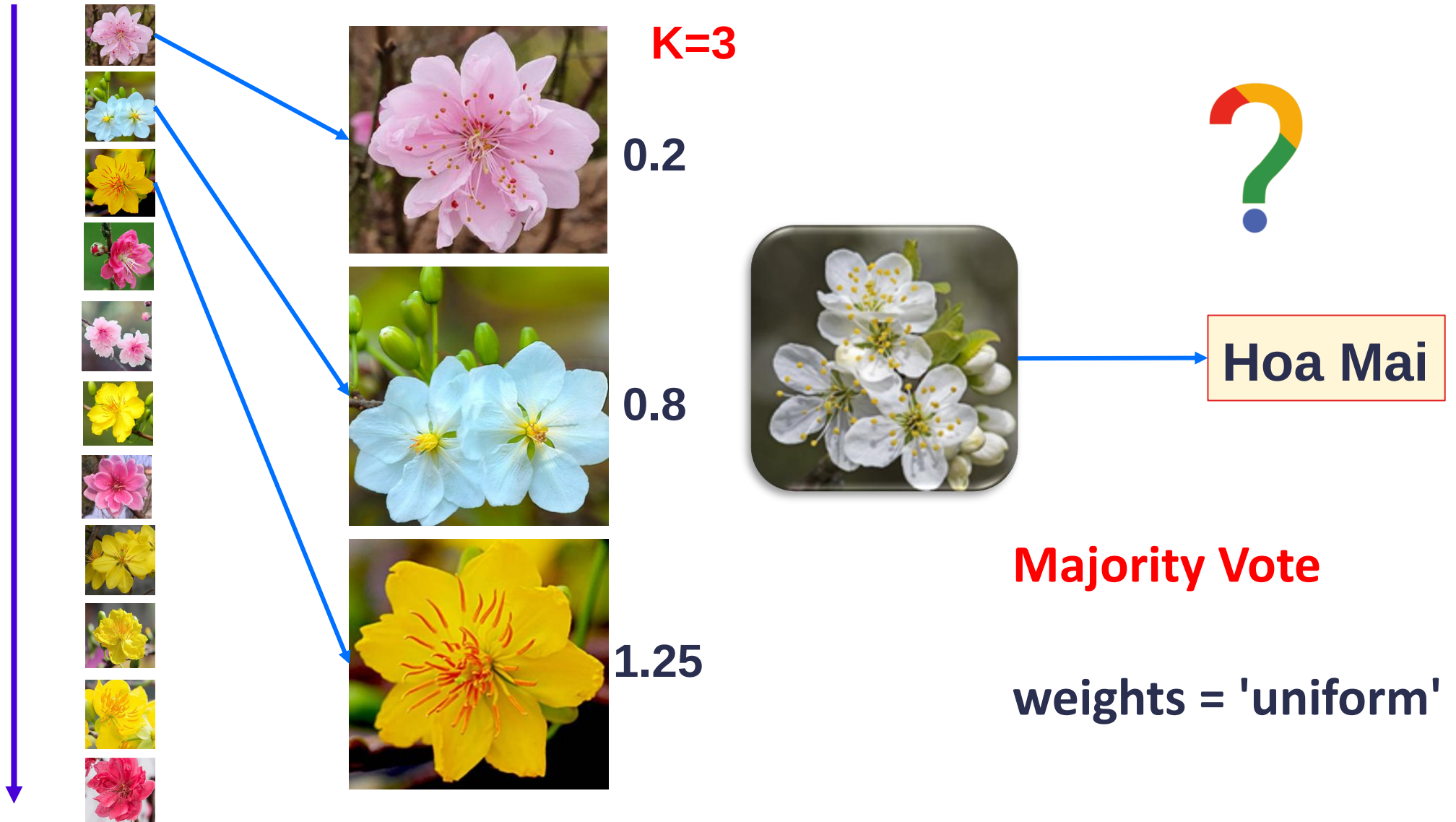


Phương pháp xác định kết quả

Sử dụng cơ chế bình chọn theo số đông (Majority Vote)

- Xem tất cả các điểm lân cận có giá trị như nhau
- Nếu trong k láng giềng, **nhãn nào chiếm đa số** thì x sẽ được gán theo nhãn đó.

Phương pháp xác định kết quả



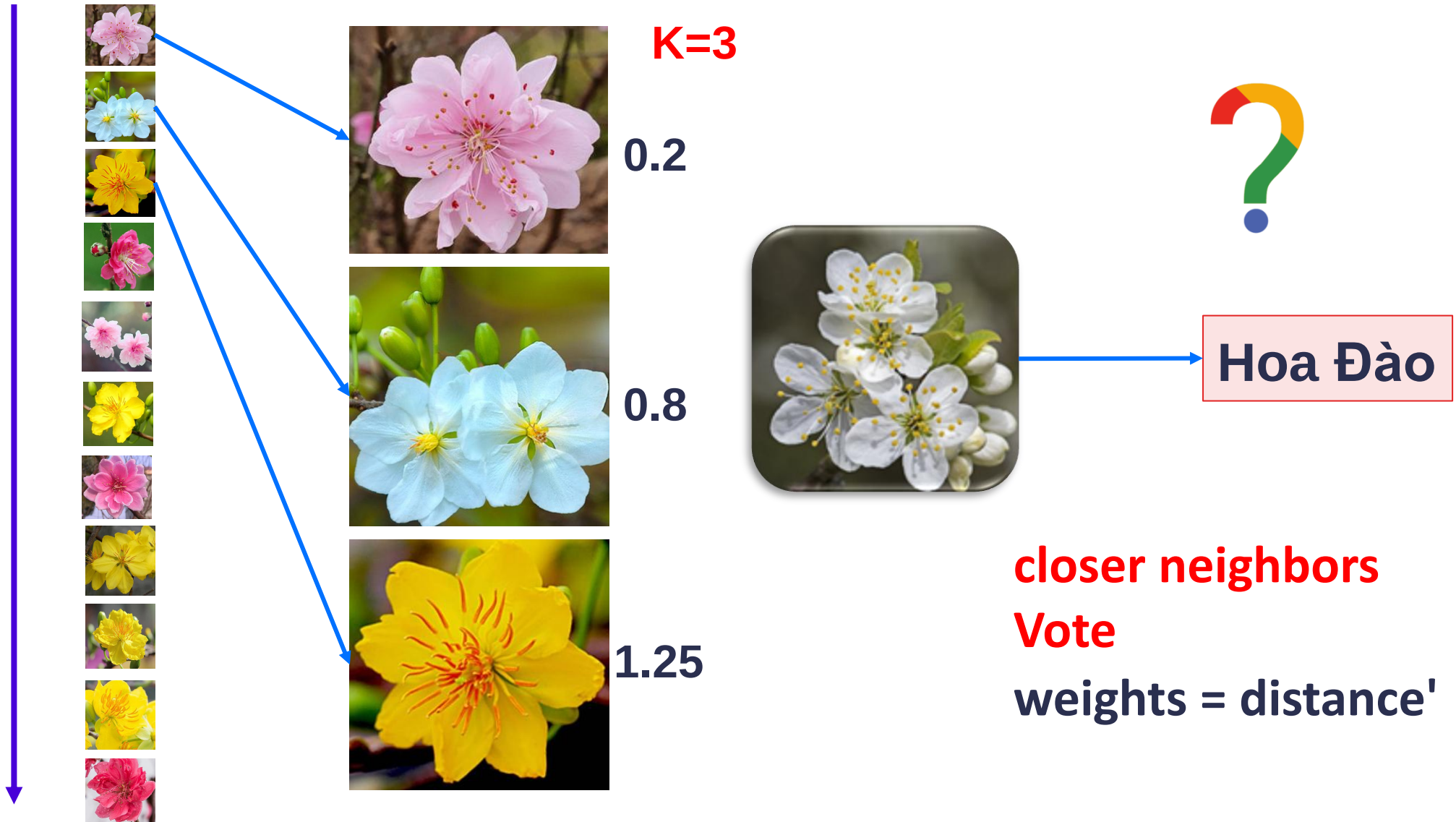


Phương pháp xác định kết quả

Sử dụng cơ chế Ưu tiên điểm lân cận

- Một điểm càng gần điểm test data thì phải được đánh trọng số càng cao.
- Cách đơn giản nhất là **lấy nghịch đảo của khoảng cách**.
- Trường hợp test data trùng với 1 điểm dữ liệu trong training data, tức khoảng cách bằng 0, ta lấy luôn label của điểm training data.

Phương pháp xác định kết quả





Chọn giá trị k

- Khi K quá nhỏ: mô hình bị ảnh hưởng bởi nhiễu dữ liệu.
- Khi K quá lớn: mô hình không nhạy với các đặc điểm cụ thể của dữ liệu.
- Thường chọn $K = \text{số lượng lớp} + 1$



Chọn giá trị k

- Cách chọn K tối ưu:
 - Dùng phương pháp cross-validation.
 - Chạy thử với nhiều giá trị K và đánh giá bằng độ chính xác trên tập kiểm tra.

Chọn giá trị k

Fei-Fei Li, Yunzhu Li, Ruohan Gao

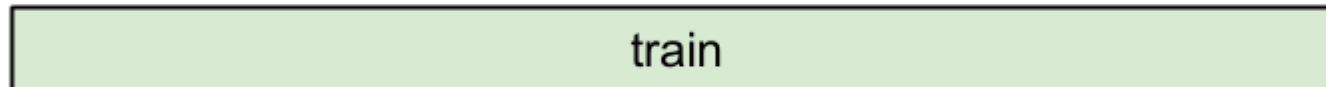
Lecture 2 - 38

April 6, 2023

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the **training data**

BAD: $K = 1$ always works perfectly on training data



Idea #2: choose hyperparameters that work best on **test** data

BAD: No idea how algorithm will perform on new data



Idea #3: Split data into **train, val**; choose hyperparameters on val and evaluate on test

Better!



Chọn giá trị k

Fei-Fei Li, Yunzhu Li, Ruohan Gao

Lecture 2 - 39

April 6, 2023

Setting Hyperparameters

train

Idea #4: Cross-Validation: Split data into **folds**, try each fold as validation and average the results

fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test

Useful for small datasets, but not used too frequently in deep learning

Tăng tốc cho KNN

- Brute Force: tính toán khoảng cách từ một điểm test data đến tất cả các điểm trong training set
- Tăng tốc việc tìm kiếm: K-D Tree và Ball Tree.

algorithm : {'auto', 'ball_tree', 'kd_tree', 'brute'}, default='auto'

Algorithm used to compute the nearest neighbors:

- 'ball_tree' will use `BallTree`
- 'kd_tree' will use `KDTree`
- 'brute' will use a brute-force search.
- 'auto' will attempt to decide the most appropriate algorithm based on the values passed to `fit` method.



Ưu điểm và nhược điểm của KNN

Ưu điểm:

- Dễ hiểu, dễ triển khai.
- Không cần huấn luyện mô hình
- Độ phức tạp tính toán của quá trình training là bằng 0.



Ưu điểm và nhược điểm của KNN

Nhược điểm:

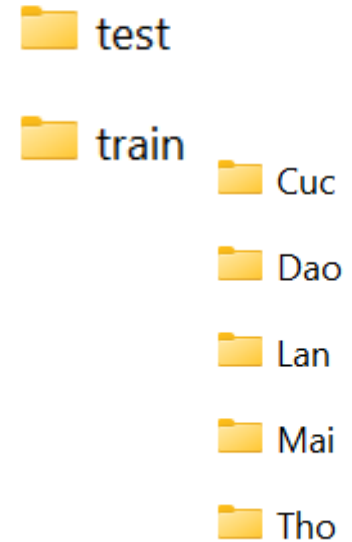
- Tốc độ chậm với dữ liệu lớn.
- Nhạy cảm với nhiễu.
- Phụ thuộc vào cách chọn khoảng cách và cách biểu diễn đặc trưng.



Áp dụng cho bài toán phân loại ảnh

Tập dữ liệu

- Giả sử ta có tập dữ liệu: HoaVietNam
- Có 4 loài hoa:
 - Cuc – Hoa cúc
 - Dao – Hoa đào
 - Lan – Hoa phong lan
 - Mai – Hoa mai
 - Tho – Hoa vạn thọ
- Đã chia thành 2 phần: train và test



Tập dữ liệu

Tập train: Cuc – Hoa cúc



0001.jpg



0002.jpg



0003.jpg



0004.jpg



0005.jpg



0006.jpg



0007.jpg



0008.jpg



0009.jpg



0010.jpg



0011.jpg



0012.jpg



0013.jpg



0014.jpg



0015.jpg



0016.jpg



0017.jpg



0018.jpg



0019.jpg



0020.jpg



0021.jpg



0022.jpg



0023.jpg



0024.jpg



0025.jpg



0026.jpg



0027.jpg



0028.jpg



0029.jpg



0030.jpg

Tập dữ liệu

Tập train: Dao – Hoa đào



0001.jpg



0002.jpg



0003.jpg



0004.jpg



0005.jpg



0006.jpg



0007.jpg



0008.jpg



0009.jpg



0010.jpg



0011.jpg



0012.jpg



0013.jpg



0014.jpg



0015.jpg



0016.jpg



0017.jpg



0018.jpg



0019.jpg



0020.jpg



0021.jpg



0022.jpg



0023.jpg



0024.jpg



0025.jpg



0026.jpg



0027.jpg



0028.jpg



0029.jpg



0030.jpg

Tập dữ liệu

Tập train: Lan – Hoa phong lan



Tập dữ liệu

Tập train: Mai – Hoa mai



Tập dữ liệu

Tập train: Tho – Hoa vạn thọ



Tập dữ liệu

Tập test: Lan – Hoa phong lan



0031.jpg



0032.jpg



0033.jpg



0034.jpg



0035.jpg



0036.jpg



0037.jpg



0038.jpg



0039.jpg



0040.jpg



Rút trích đặc trưng cho ảnh

Nhận xét:

- Quan sát → các loài hoa này có màu khác nhau
- → Sử dụng đặc trưng Histogram màu



Sử dụng Histogram làm vector đặc trưng

```
"""Trích xuất histogram màu trong không gian RGB."""
```

```
def extract_color_histogram_rgb(image_path, bins=256):
```

```
    image = cv2.imread(image_path)    # Đọc ảnh
```

```
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Chuyển từ BGR sang RGB
```

```
    # Tính histogram và chuẩn hóa thành feature vector
```

```
    feature_vector = []
```

```
    for i in range(3): # 3 kênh màu R, G, B
```

```
        hist = cv2.calcHist([image], [i], None, [bins], [0, 256])
```

```
        hist = cv2.normalize(hist, hist).flatten() # Chuẩn hóa và làm phẳng
```

```
        feature_vector.extend(hist)
```

```
    # Chuẩn hóa và làm phẳng
```

```
    feature_vector = np.array(feature_vector)
```

```
    feature_vector = cv2.normalize(feature_vector, feature_vector).flatten()
```

```
    return feature_vector # Kích thước 256+256+256 = 768
```



Sử dụng Histogram làm vector đặc trưng

"""Đọc tất cả các ảnh và nhãn tương ứng trong thư mục folder (train, test)"""

```
def load_images_from_folder(folder):  
    images = []  
    labels = []  
    class_names = sorted(os.listdir(folder))  
  
    for label, class_name in enumerate(class_names):  
        class_folder = os.path.join(folder, class_name)  
        if os.path.isdir(class_folder):  
            for filename in os.listdir(class_folder):  
                img_path = os.path.join(class_folder, filename)  
                img = cv2.imread(img_path)  
                if img is not None:  
                    images.append(img)  
                    labels.append(label)  
  
    return images, labels, class_names
```



Rút trích đặc trưng cho tập dữ liệu

```
train_folder = "/content/drive/MyDrive/Dataset/HoaVietNam/HoaVietNam/train"
```

```
test_folder = "/content/drive/MyDrive/Dataset/HoaVietNam/HoaVietNam/test"
```

```
train_images, train_labels, class_names = load_images_from_folder(train_folder)
```

```
test_images, test_labels, _ = load_images_from_folder(test_folder)
```



Rút trích đặc trưng cho tập dữ liệu

```
X_train = np.array([compute_histogram_feature(img, bins=256) for img in train_images])  
y_train = np.array(train_labels)
```

```
X_test = np.array([compute_histogram_feature(img, bins=256) for img in test_images])  
y_test = np.array(test_labels)
```

```
print(f"Kích thước tập train: {X_train.shape}, {y_train.shape}")  
print(f"Kích thước tập test: {X_test.shape}, {y_test.shape}")
```




Huấn luyện mô hình

#. Huấn luyện mô hình KNN với $K=3$

```
knn = KNeighborsClassifier(n_neighbors=3)  
knn.fit(X_train, y_train)
```

Đánh giá kết quả

#. Dự đoán và đánh giá mô hình

```
y_pred = knn.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy * 100:.2f}%')
```



Tài liệu tham khảo

- <https://machinelearningcoban.com/2017/01/08/knn/>
- <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>



CS231. Nhập môn Thị giác máy tính

Machine Learning Image Classification: Linear classifiers

Targets :



IBM Developer

SKILLS NETWORK 

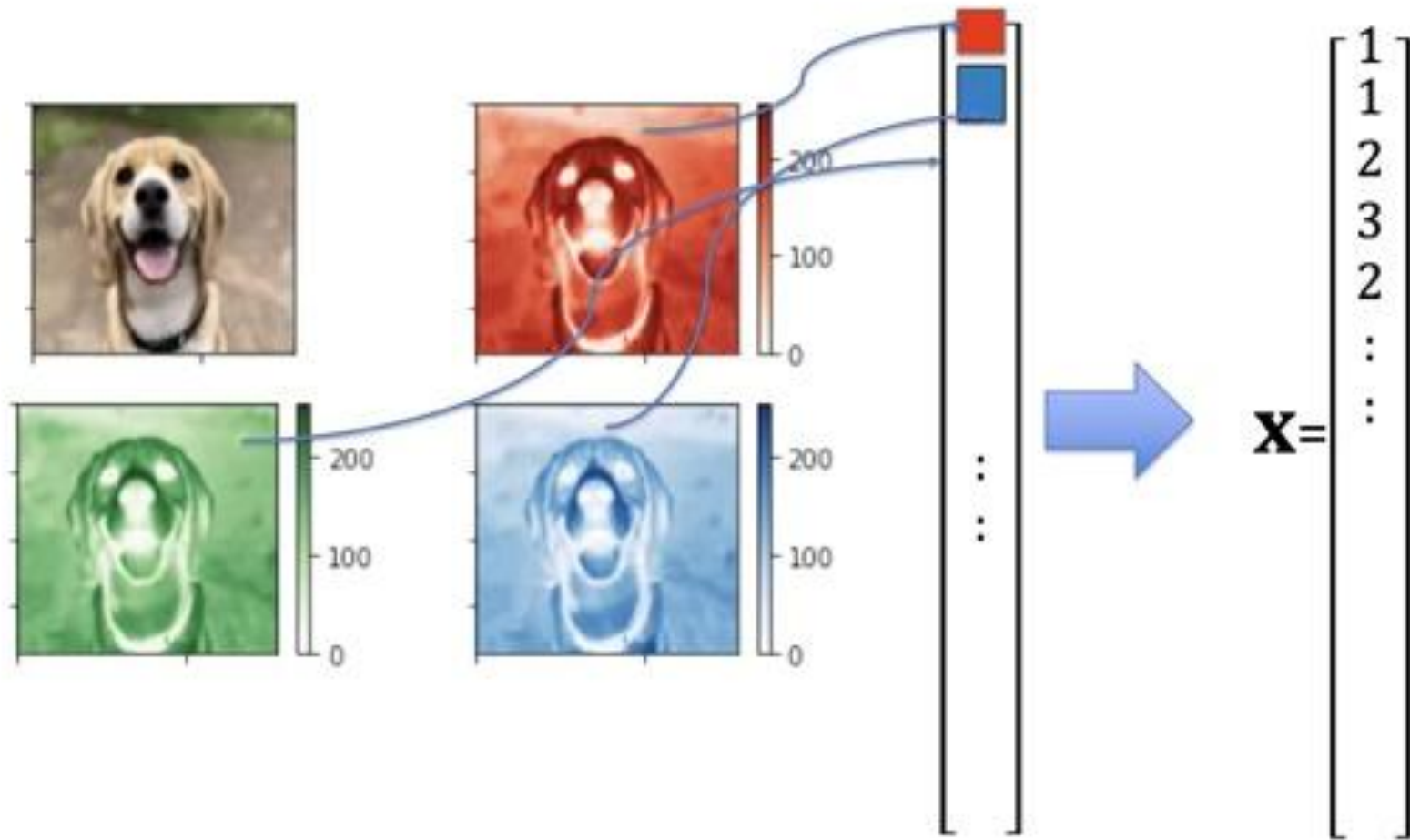
Targets :

$y = 0$



$y = 1$





Function:

$$\hat{y} = f(\text{)$$

The diagram illustrates a function f that takes an input x and produces an output $\hat{y}$. The input x is represented by a photograph of a beagle dog, which is shown with a bracket above it labeled x .

Function:

$$\hat{y} = f(\text{) = 1$$

The image shows a beagle dog with its mouth open, looking happy. A bracket above the image is labeled with the letter 'x', indicating that the image is the input to the function 'f'.

Function:

Probability this image is a dog

$$p(\text{ ) = 0.92$$



Linear Classifier

- Learnable Parameters

$$z = wx + b$$

Linear Classifier

- Learnable Parameters

$$z = wX + b$$

$$z = w_1x_1 + \dots + w_dx_d + b$$

Linear Classifier

- Learnable Parameters

$$z = wX + b$$

$$z = w_1x_1 + \dots + w_dx_d + b$$

$$z = wX + b$$



Linear Classifier

$$z = \mathbf{w}\mathbf{x} + b$$

$$z = w_1x_1 + w_2x_2 + b$$



Linear Classifier

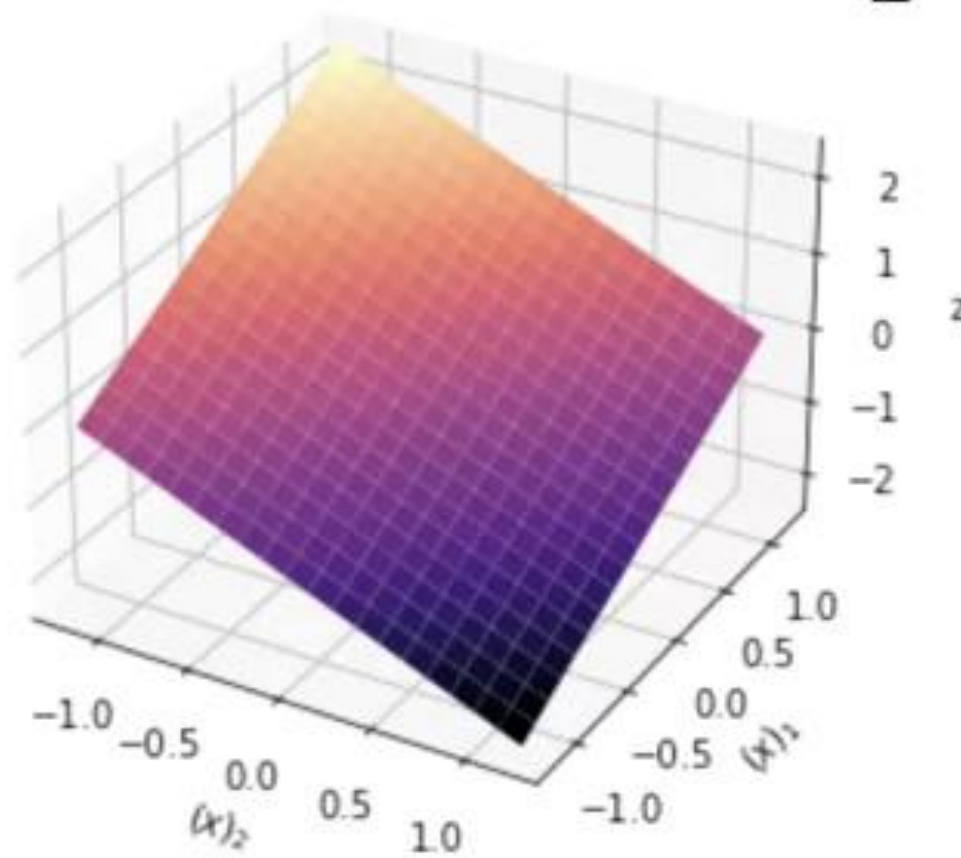
$$z = \mathbf{w}\mathbf{x} + b$$

$$z = w_1x_1 + w_2x_2 + b$$

$$z = 1x_1 - 1x_2 + 1$$

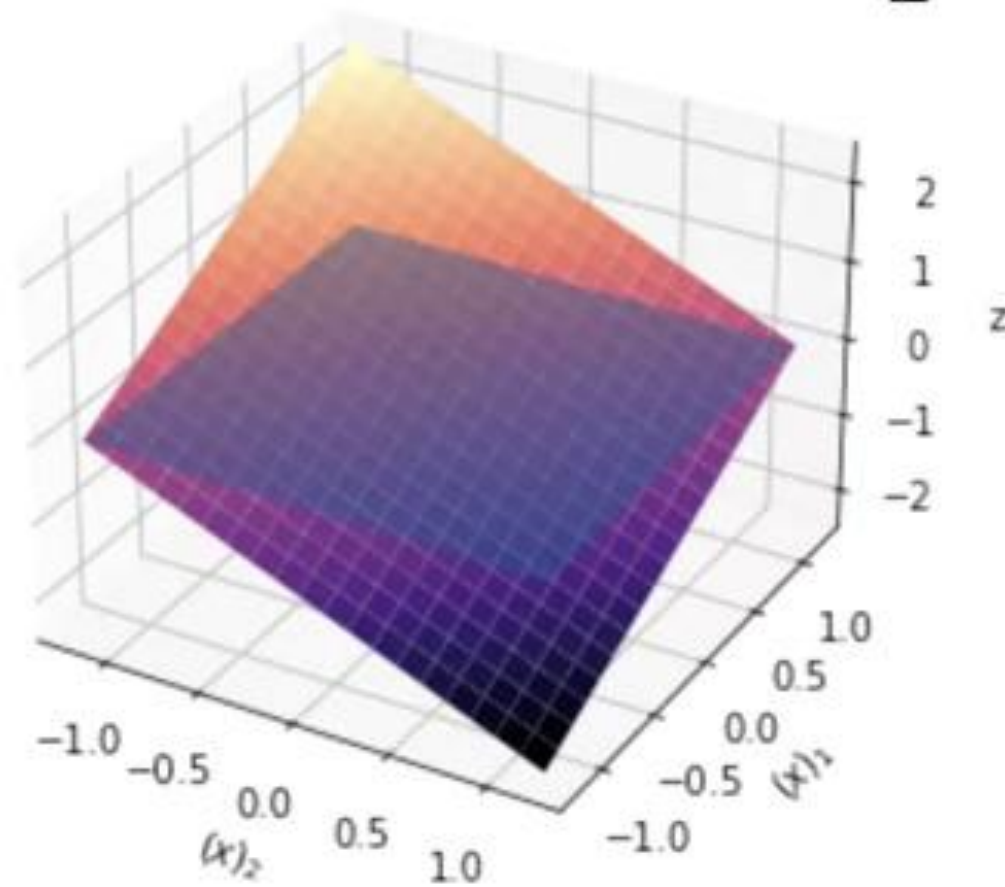
$$z = w_1x_1 + w_2x_2 + b$$

$$z = \mathbf{w}\mathbf{x} + b$$



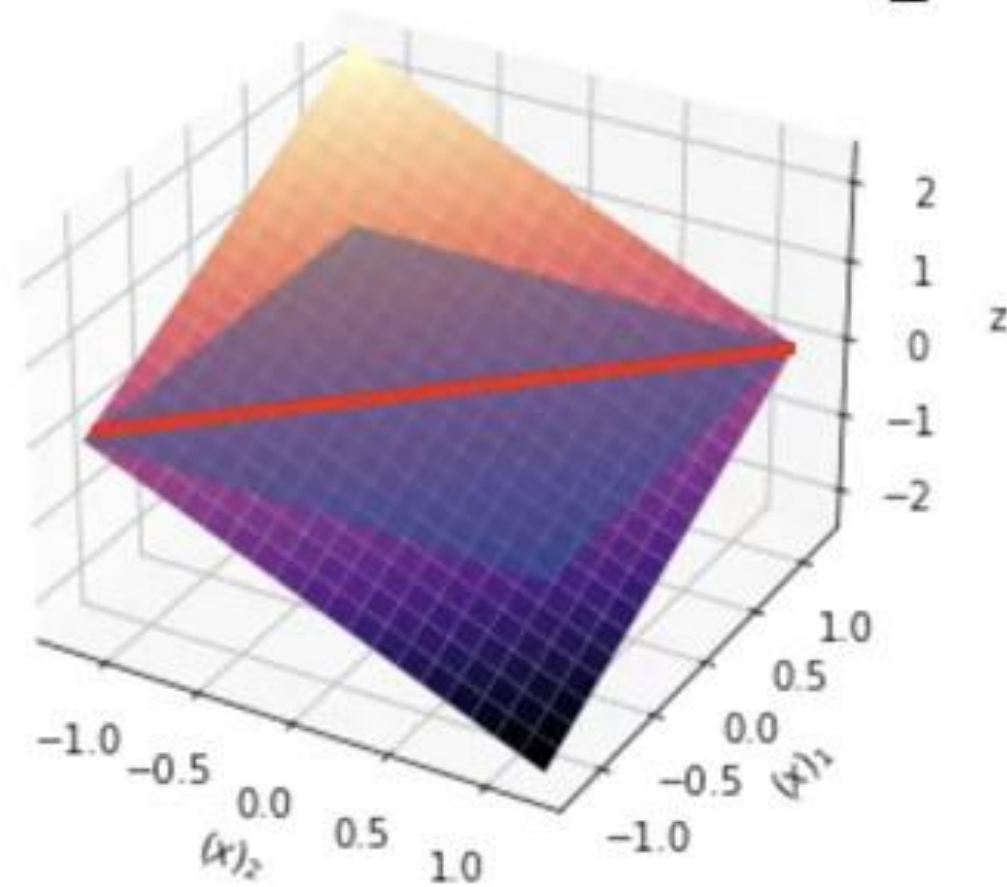
$$z = w_1x_1 + w_2x_2 + b$$

$$z = \mathbf{w}\mathbf{x} + b$$



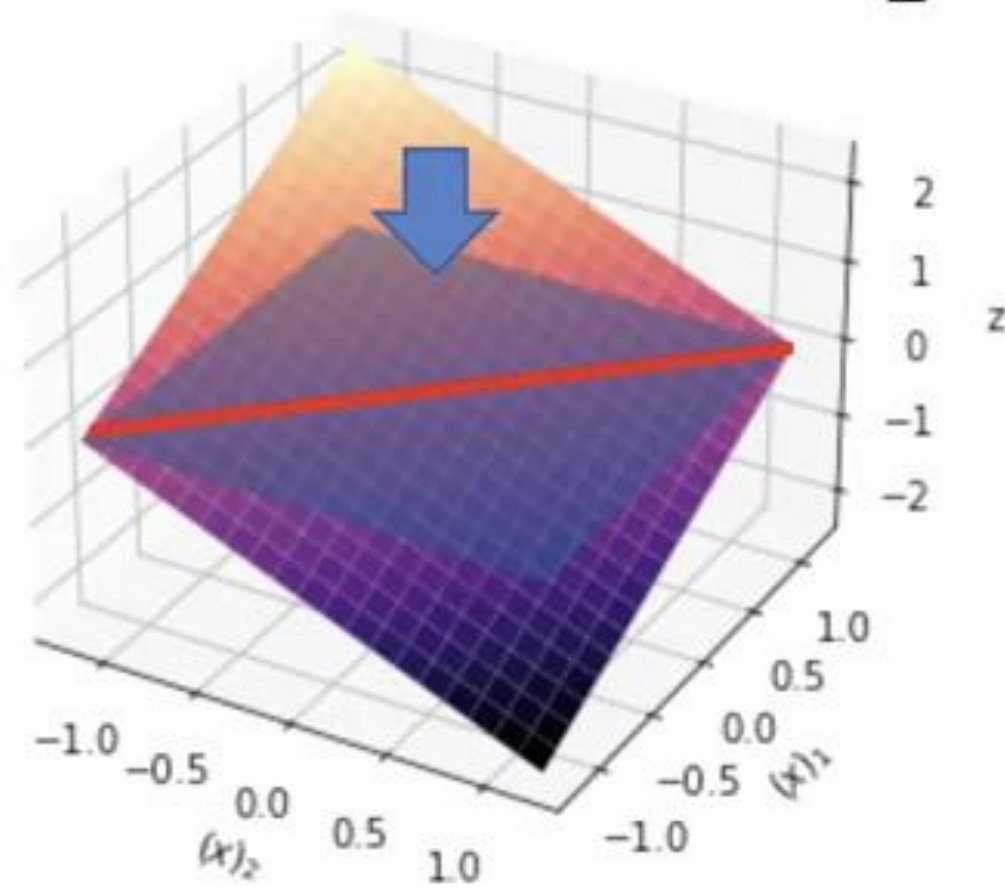
$$z = w_1x_1 + w_2x_2 + b$$

$$z = \mathbf{w}\mathbf{x} + b$$

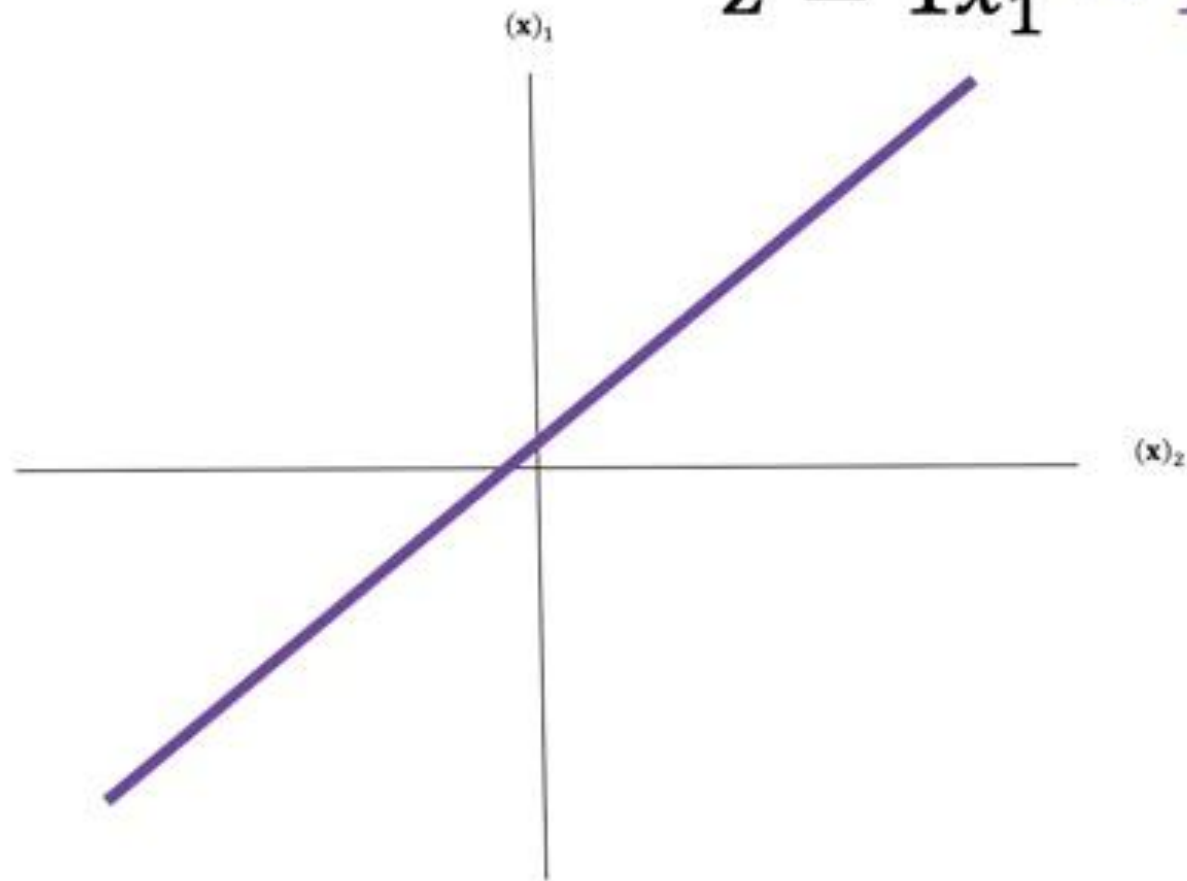


$$z = w_1x_1 + w_2x_2 + b$$

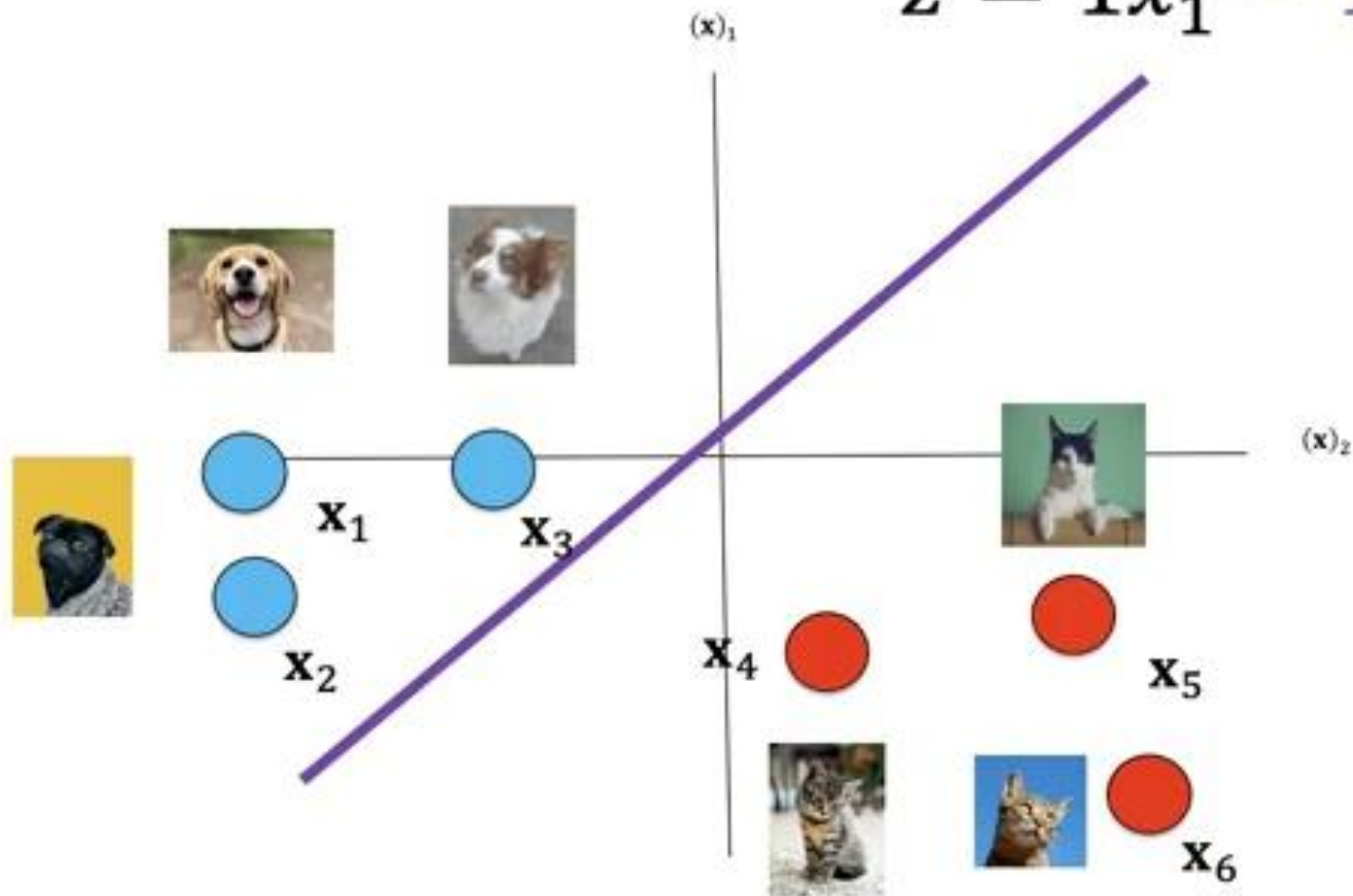
$$z = \mathbf{w}\mathbf{x} + b$$

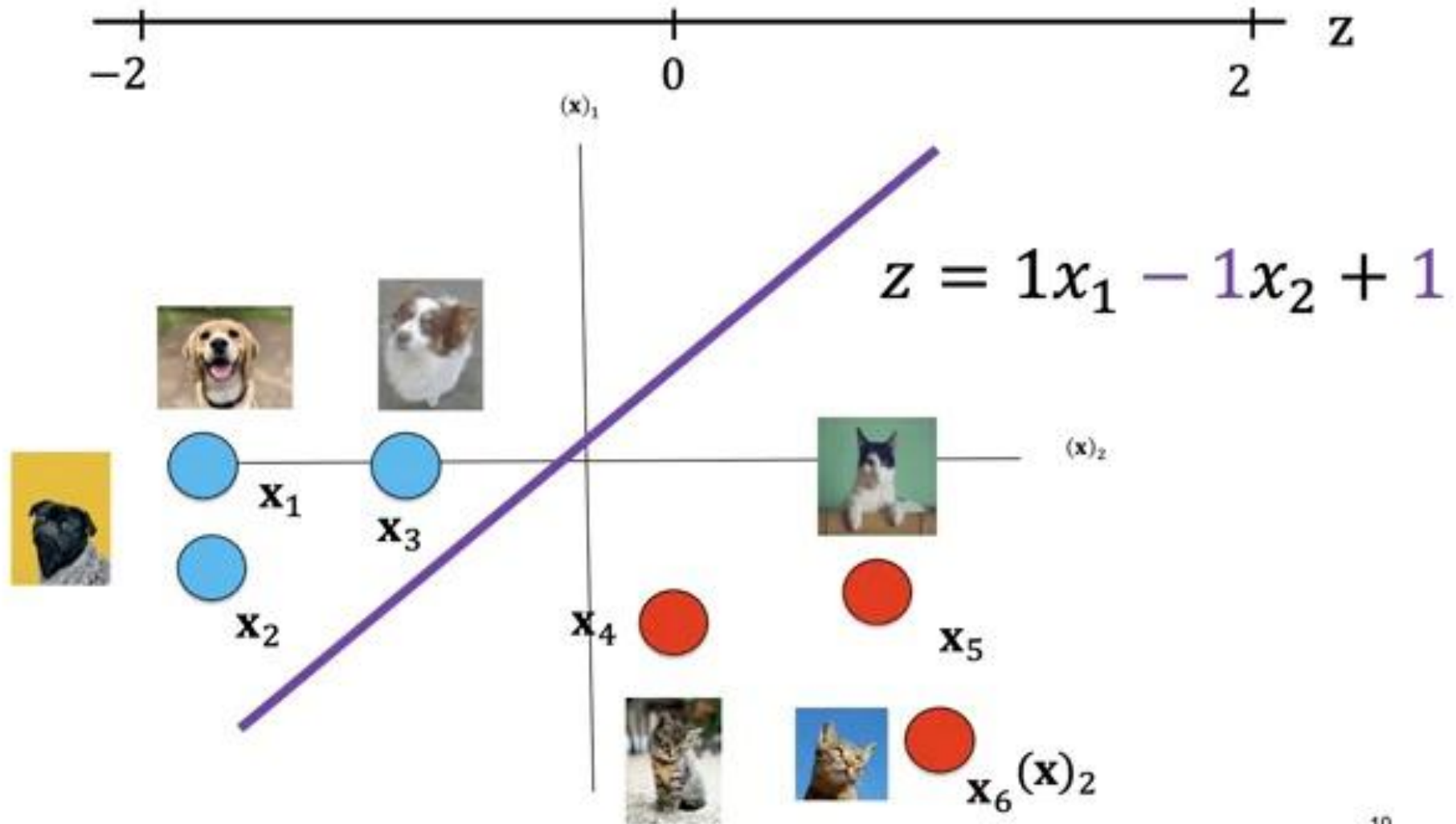


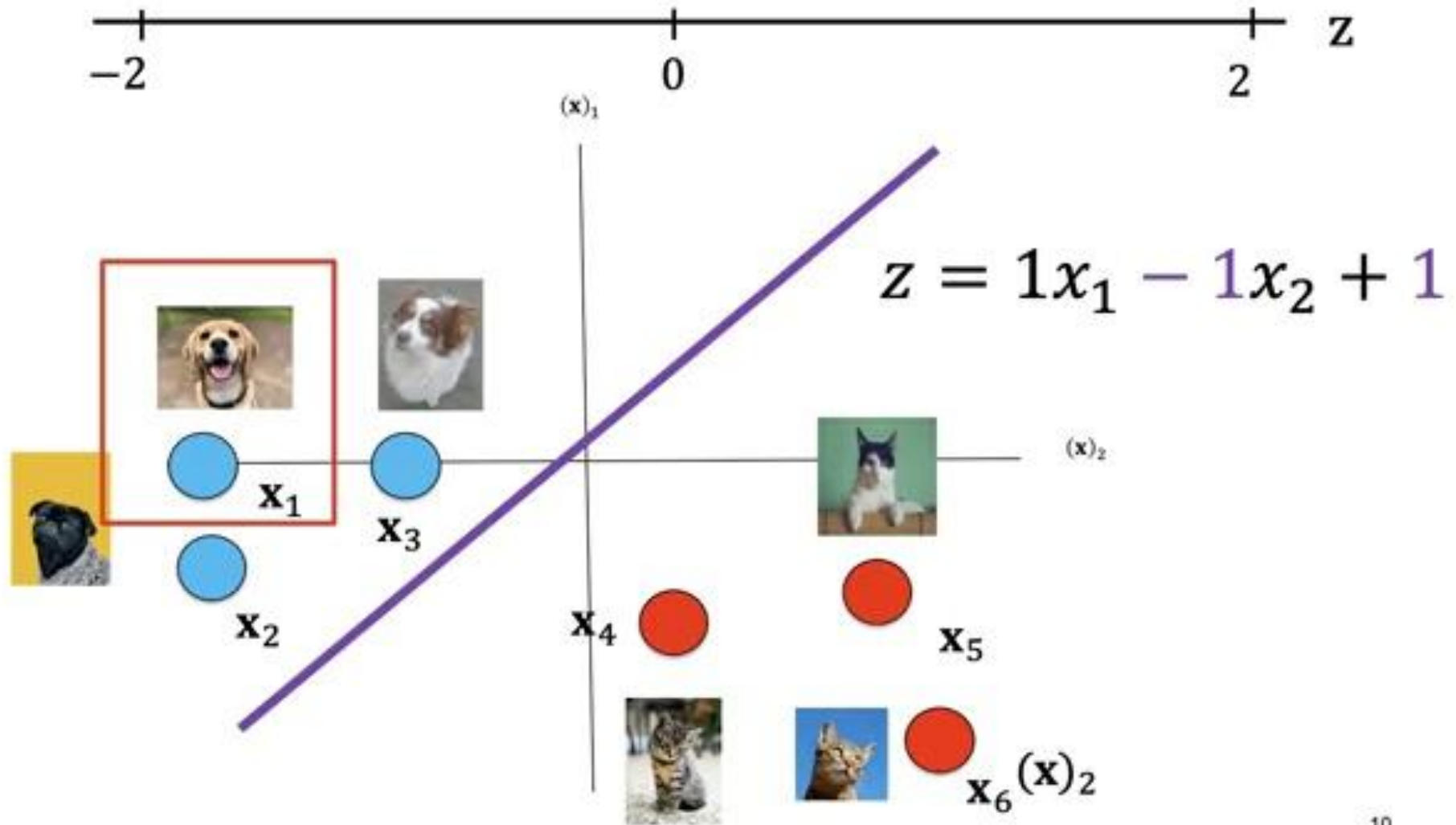
$$z = 1x_1 - 1x_2 + 1 = 0$$

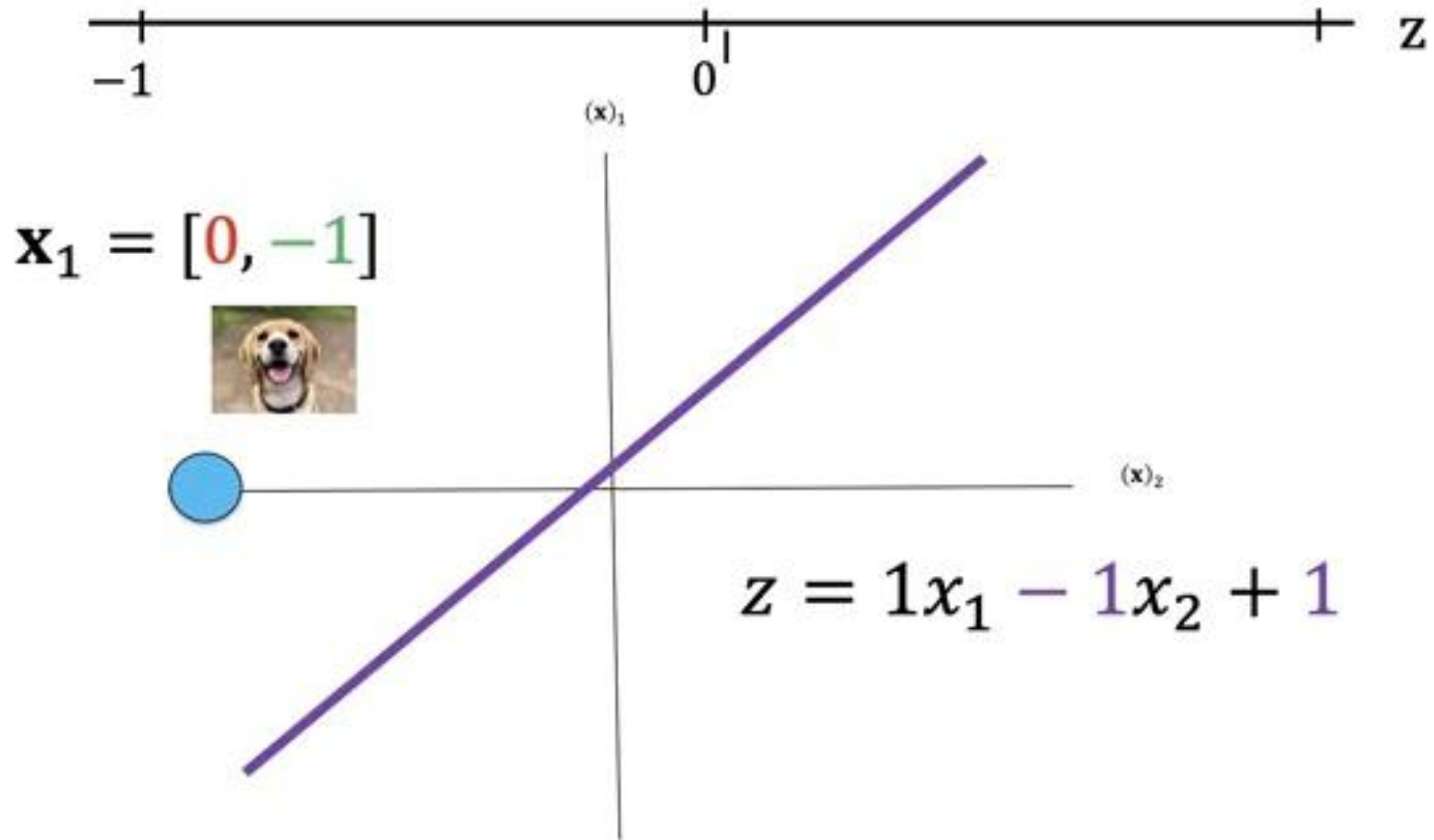


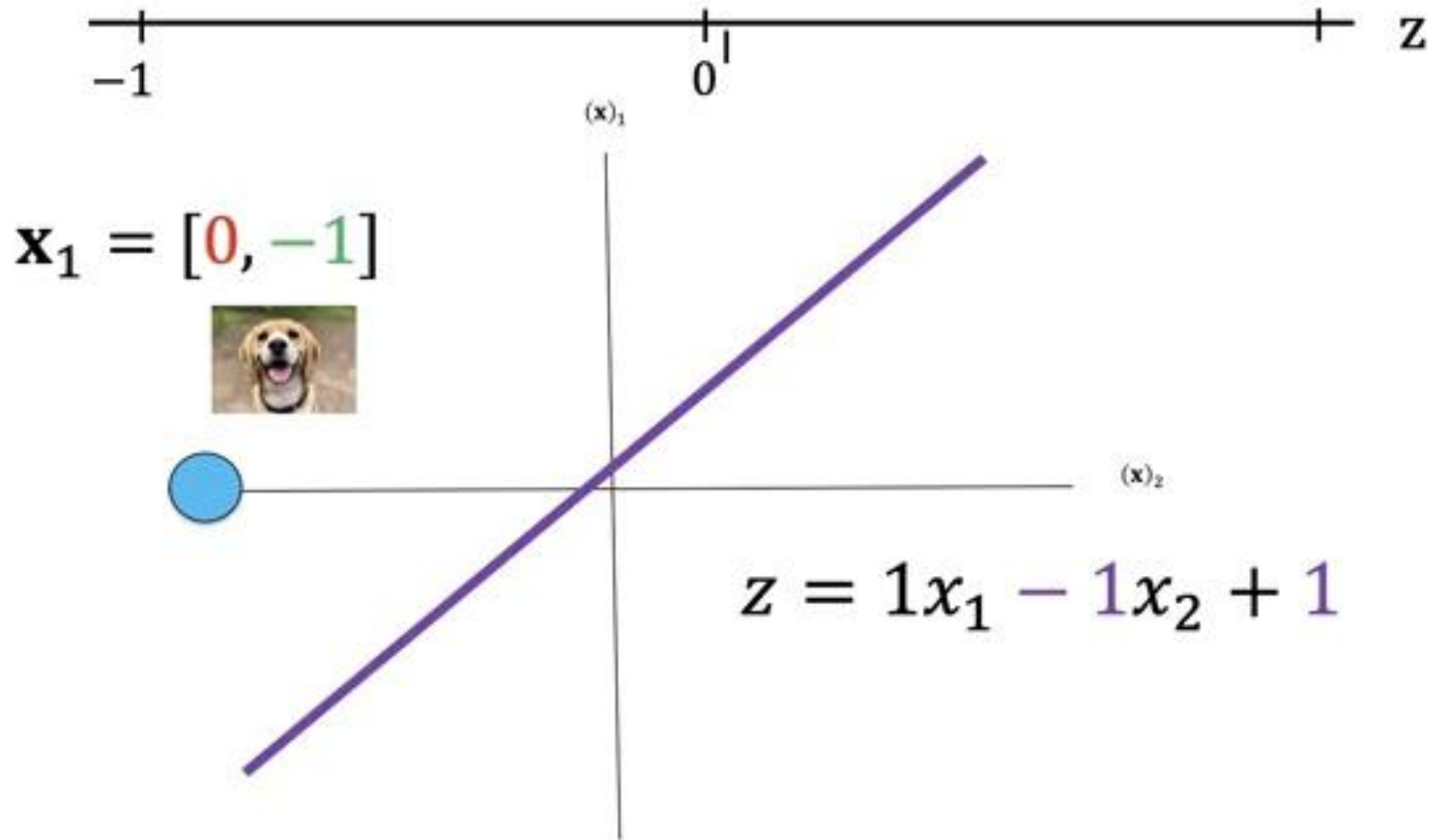
$$z = 1x_1 - 1x_2 + 1 = 0$$

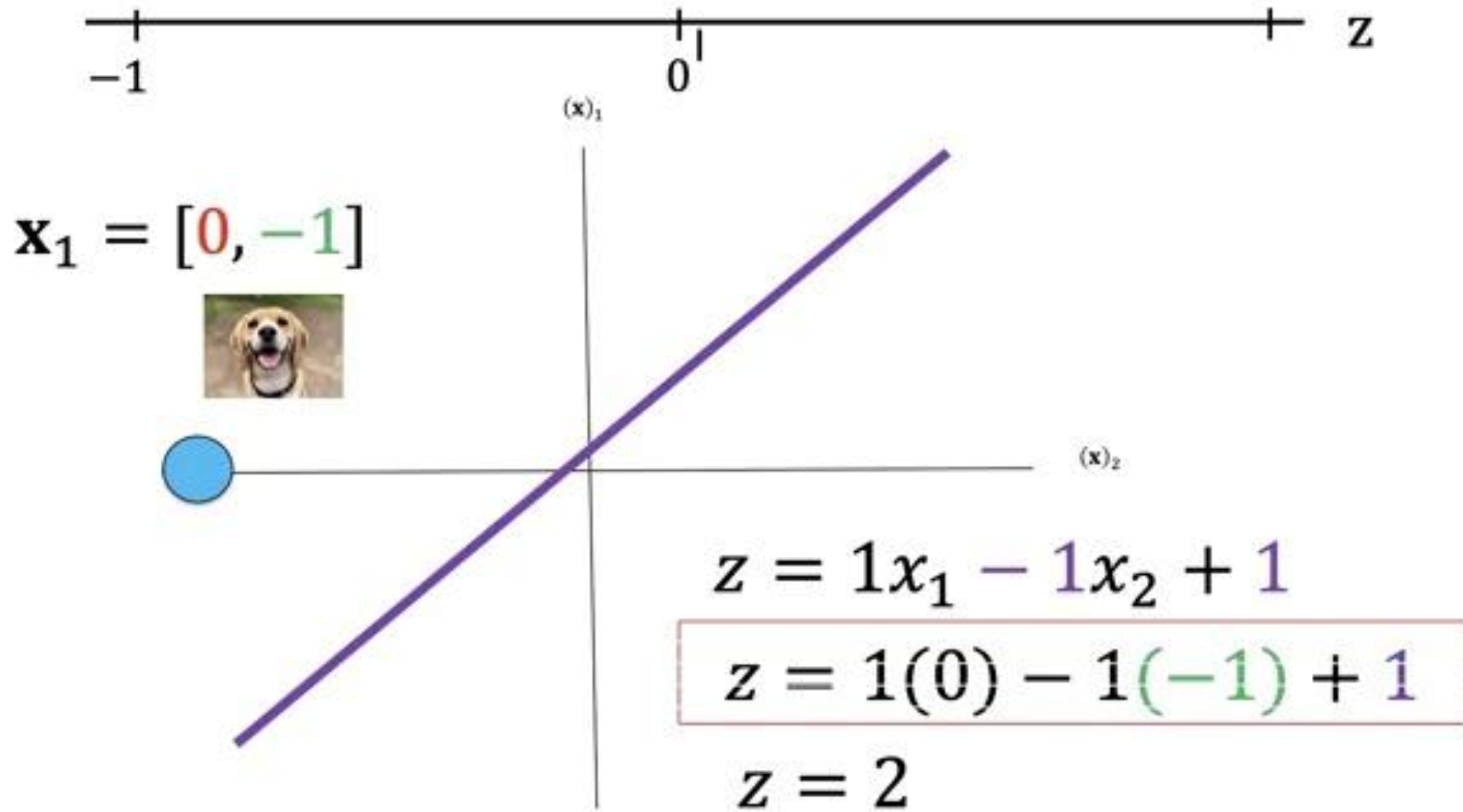


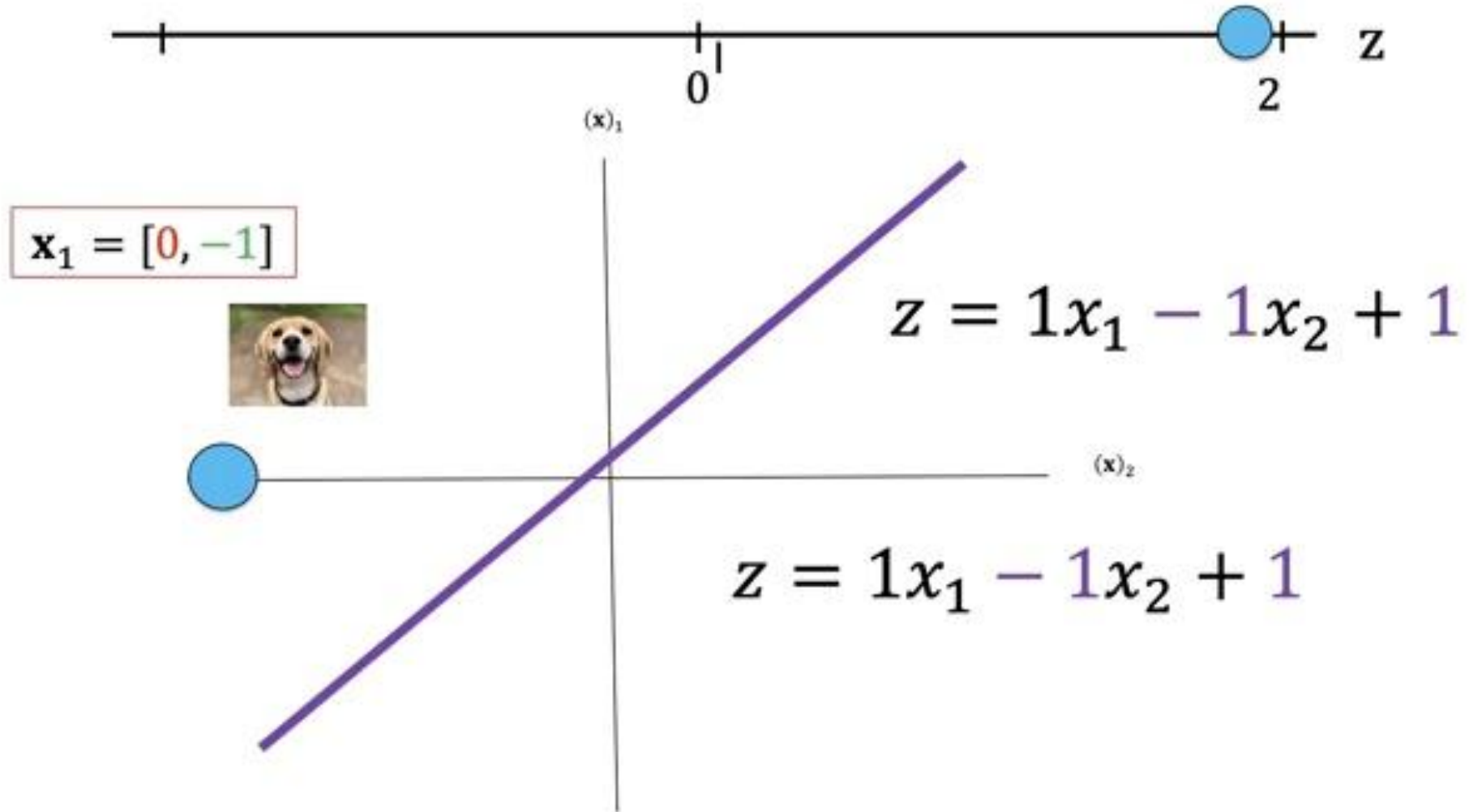


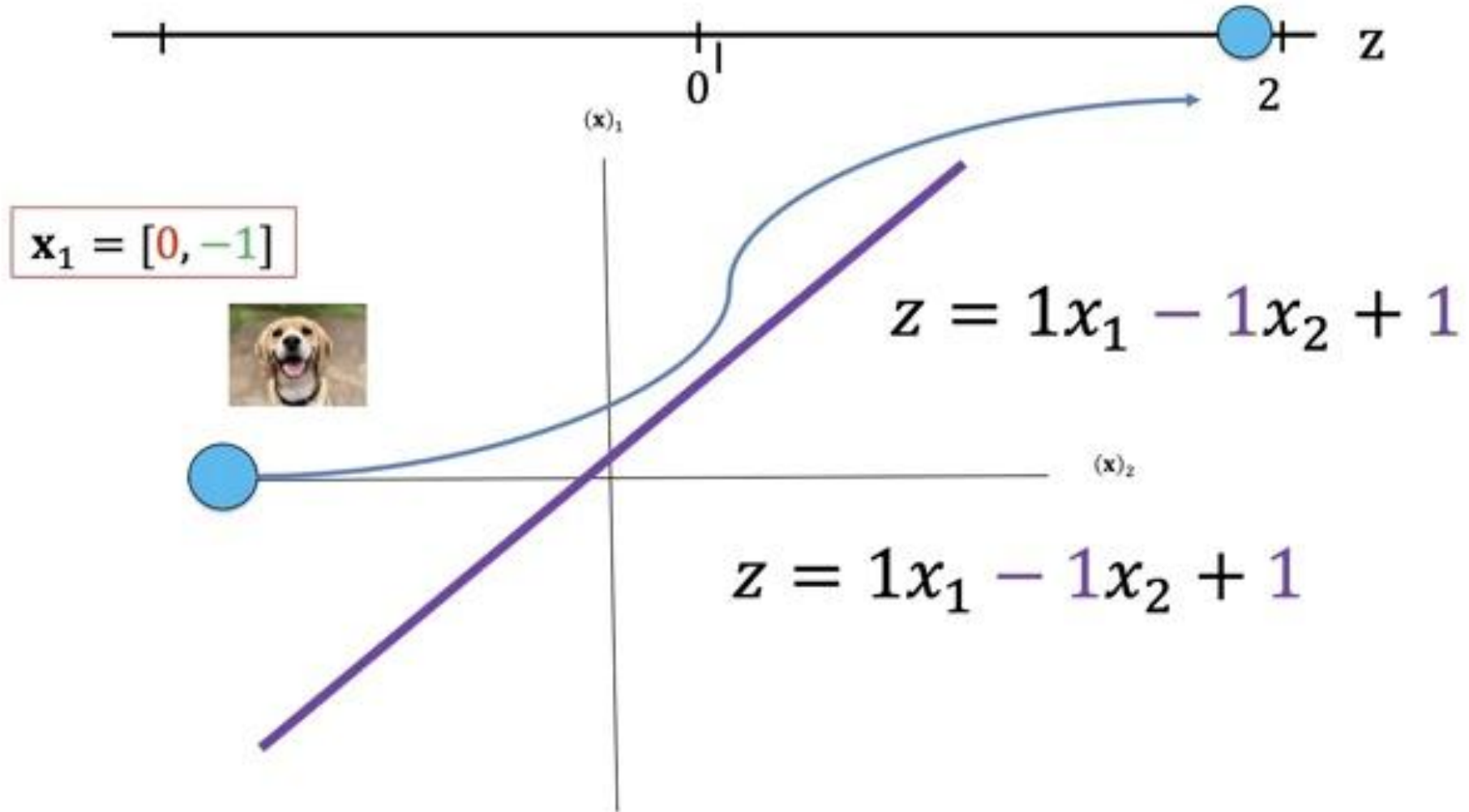


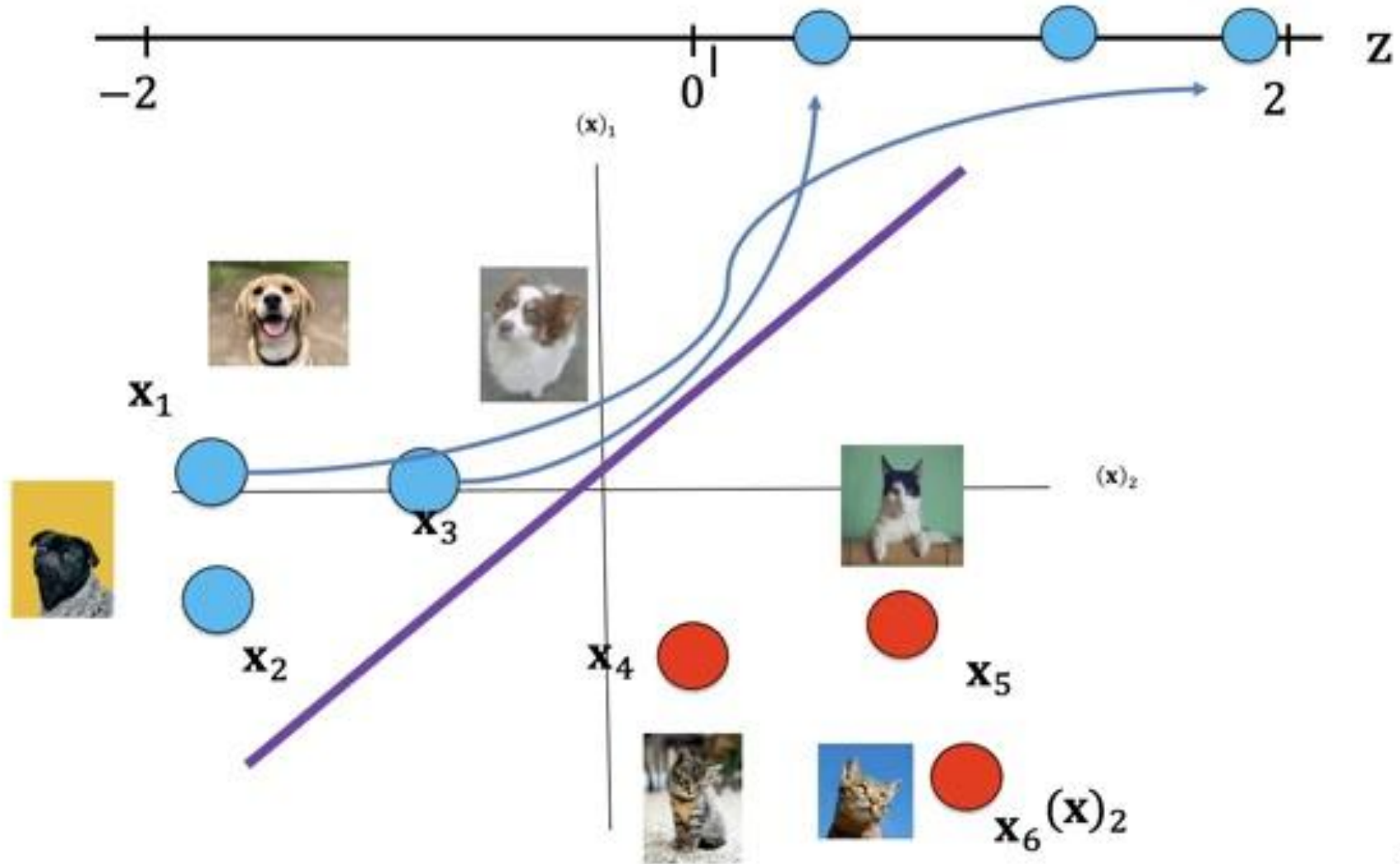


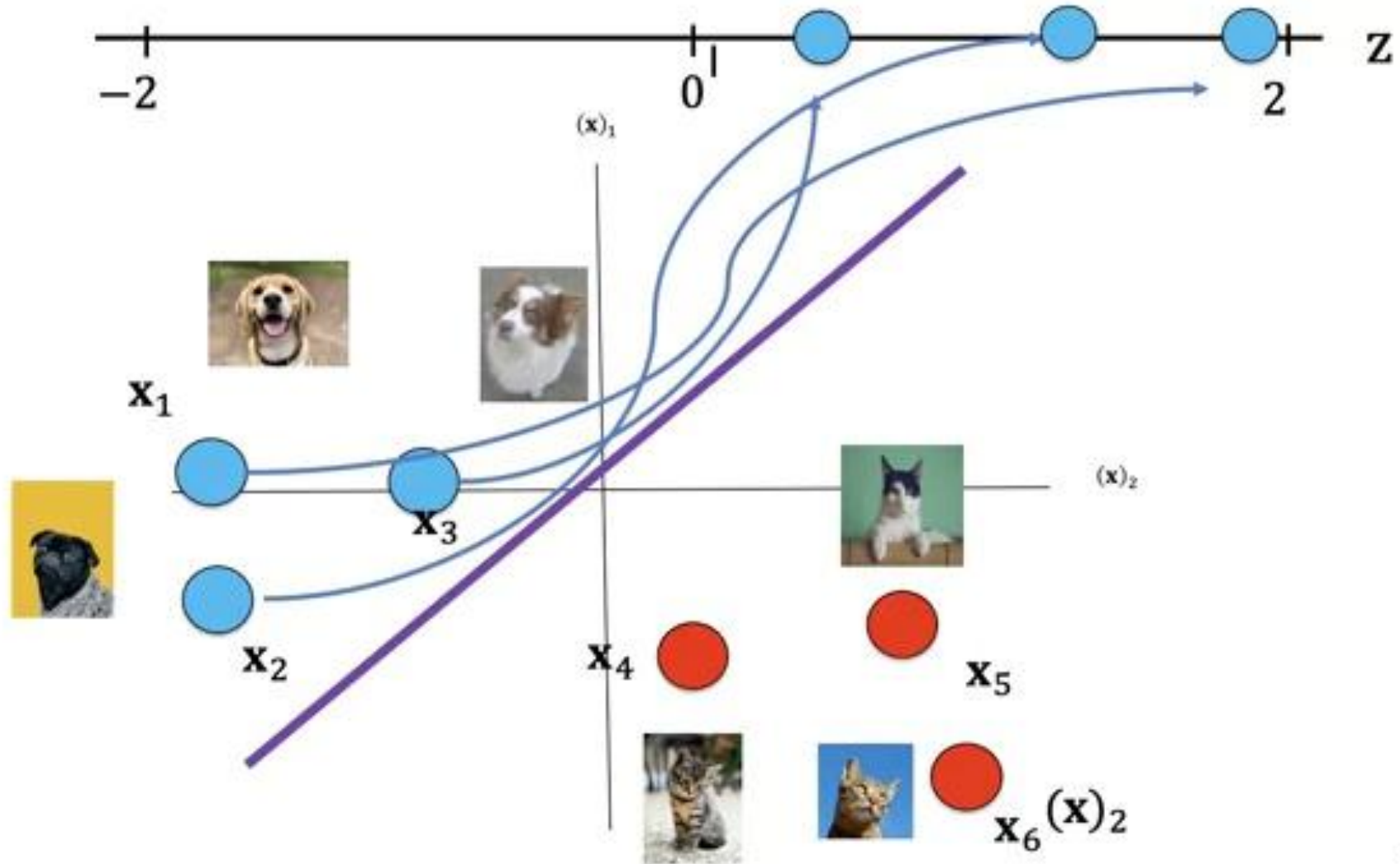


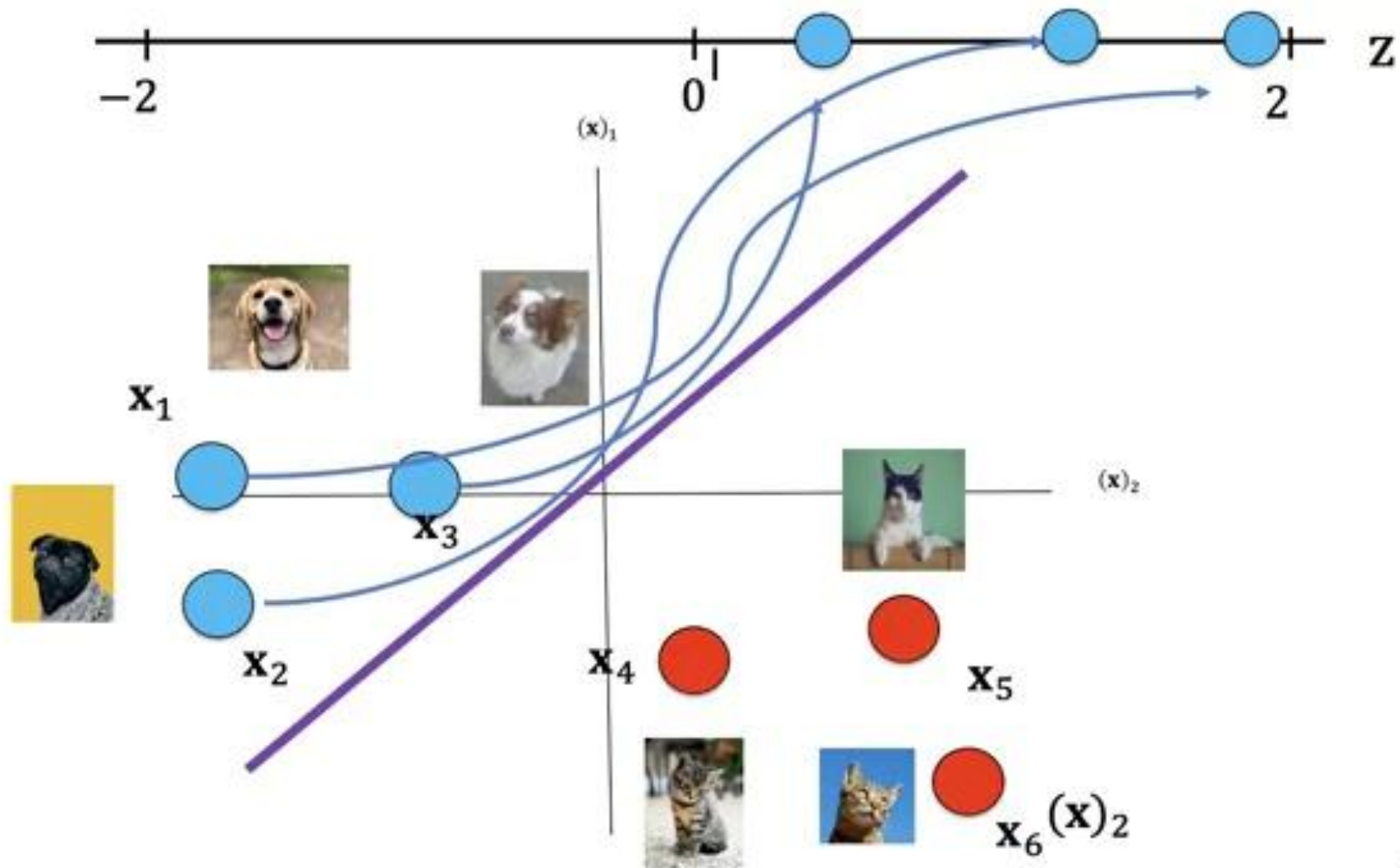


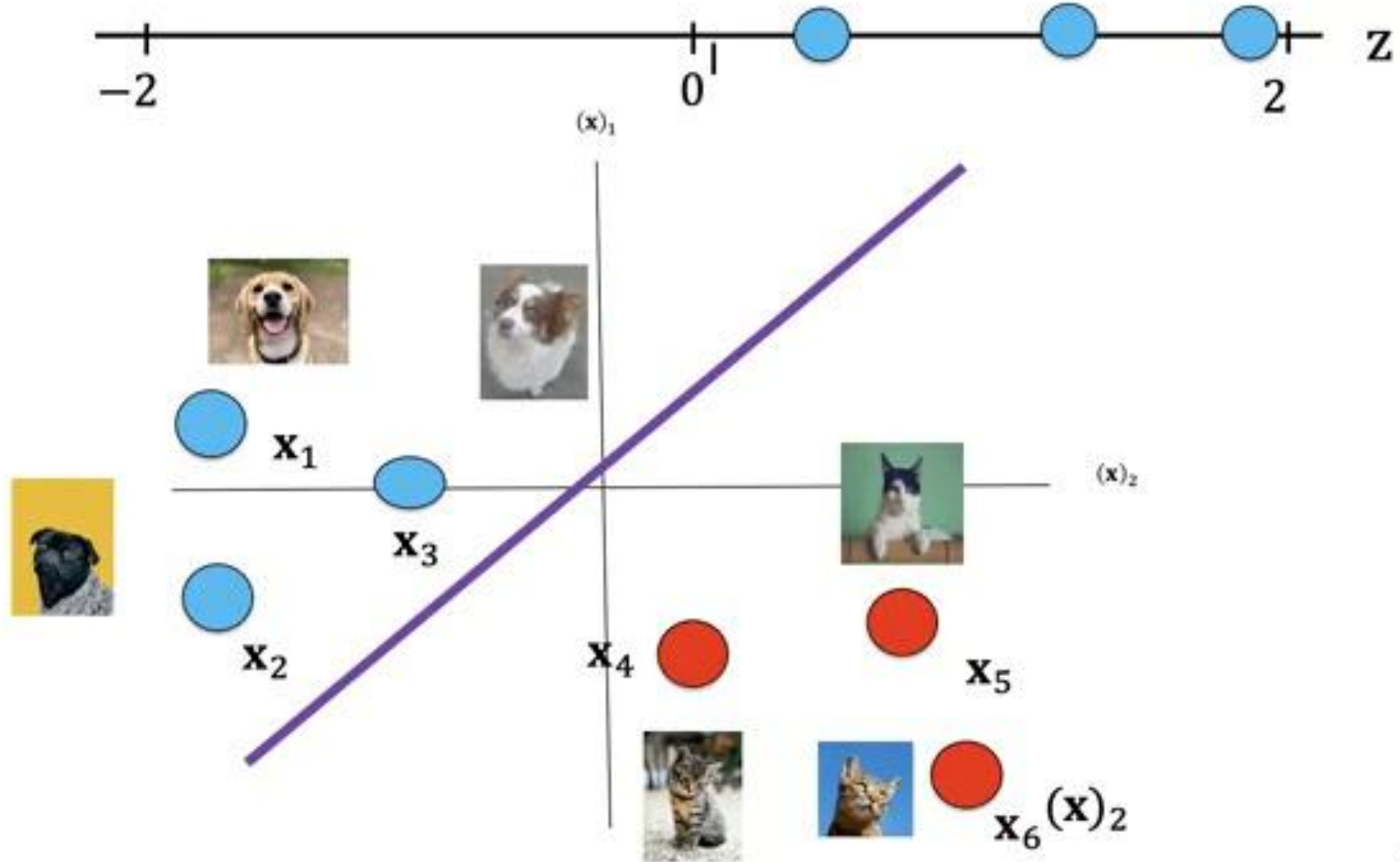


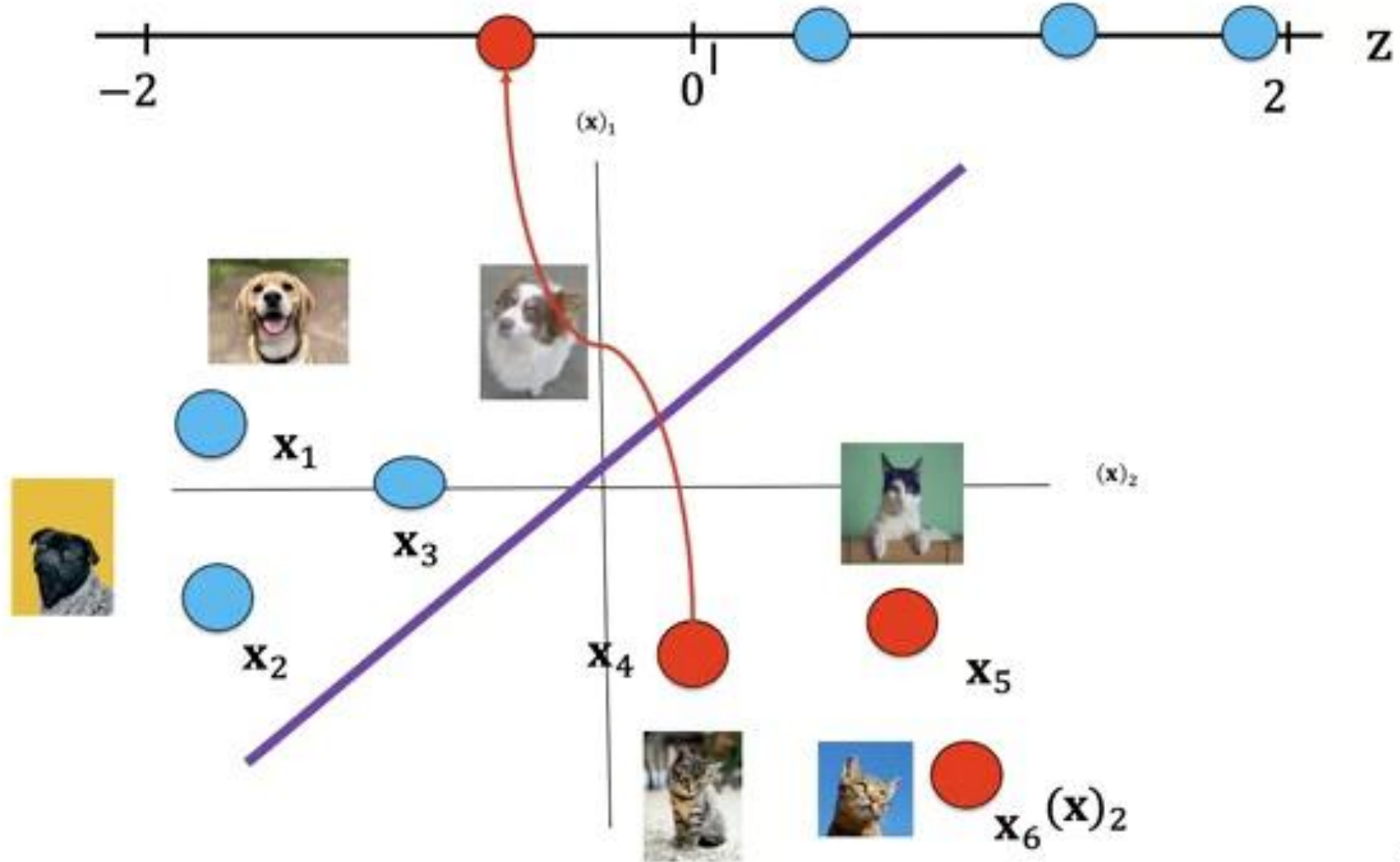


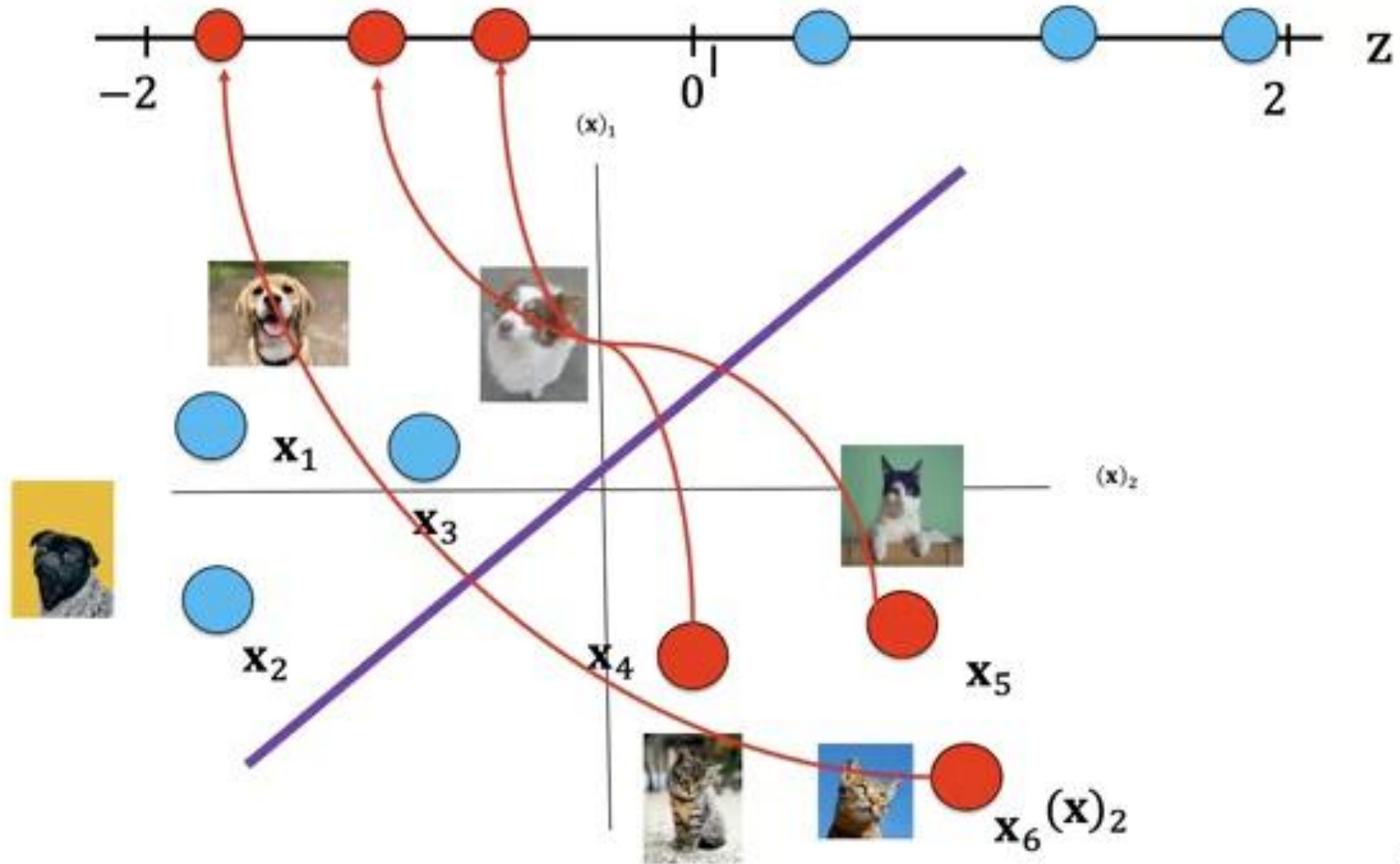




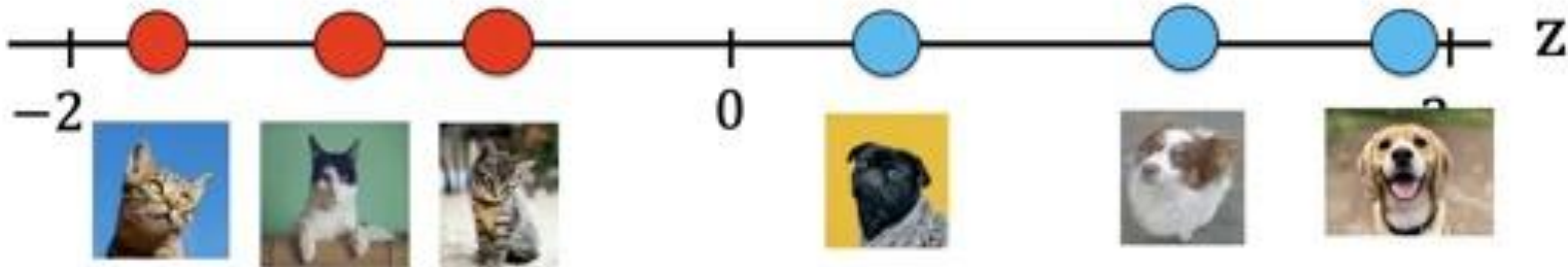




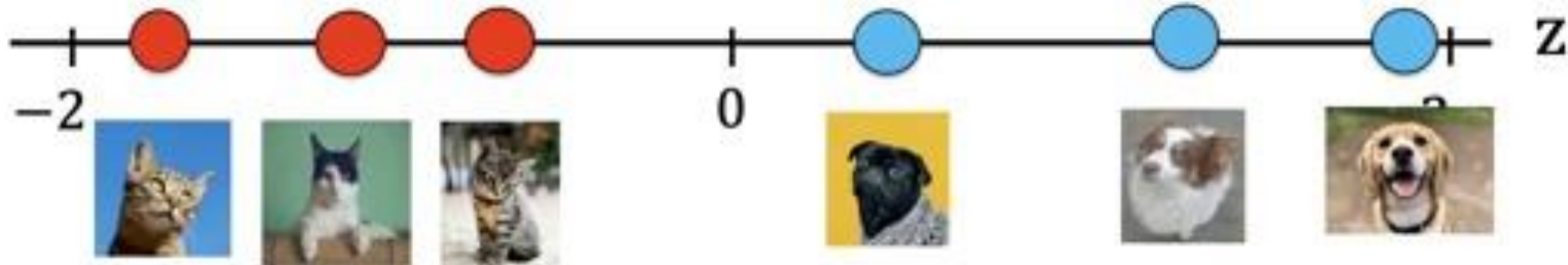




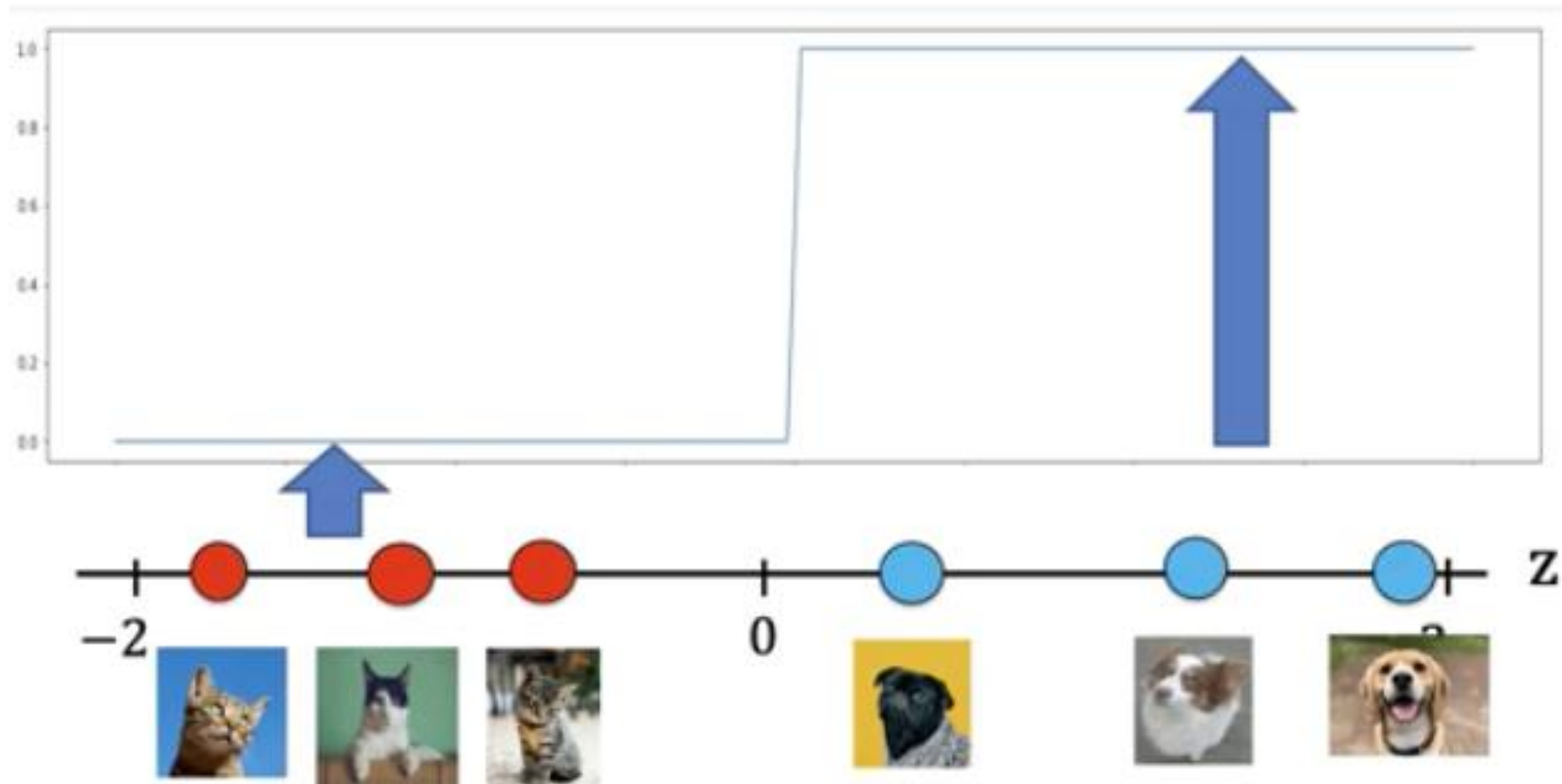
$$\hat{y} = f(z)$$



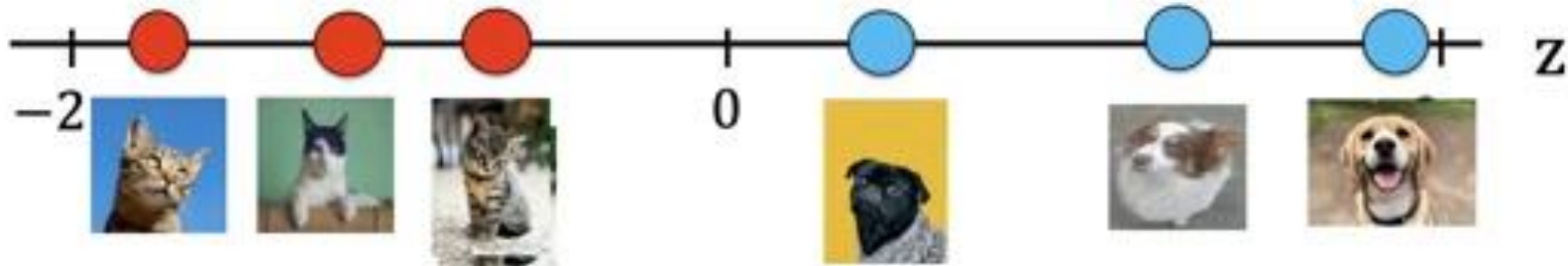
$$\hat{y} = f(z)$$



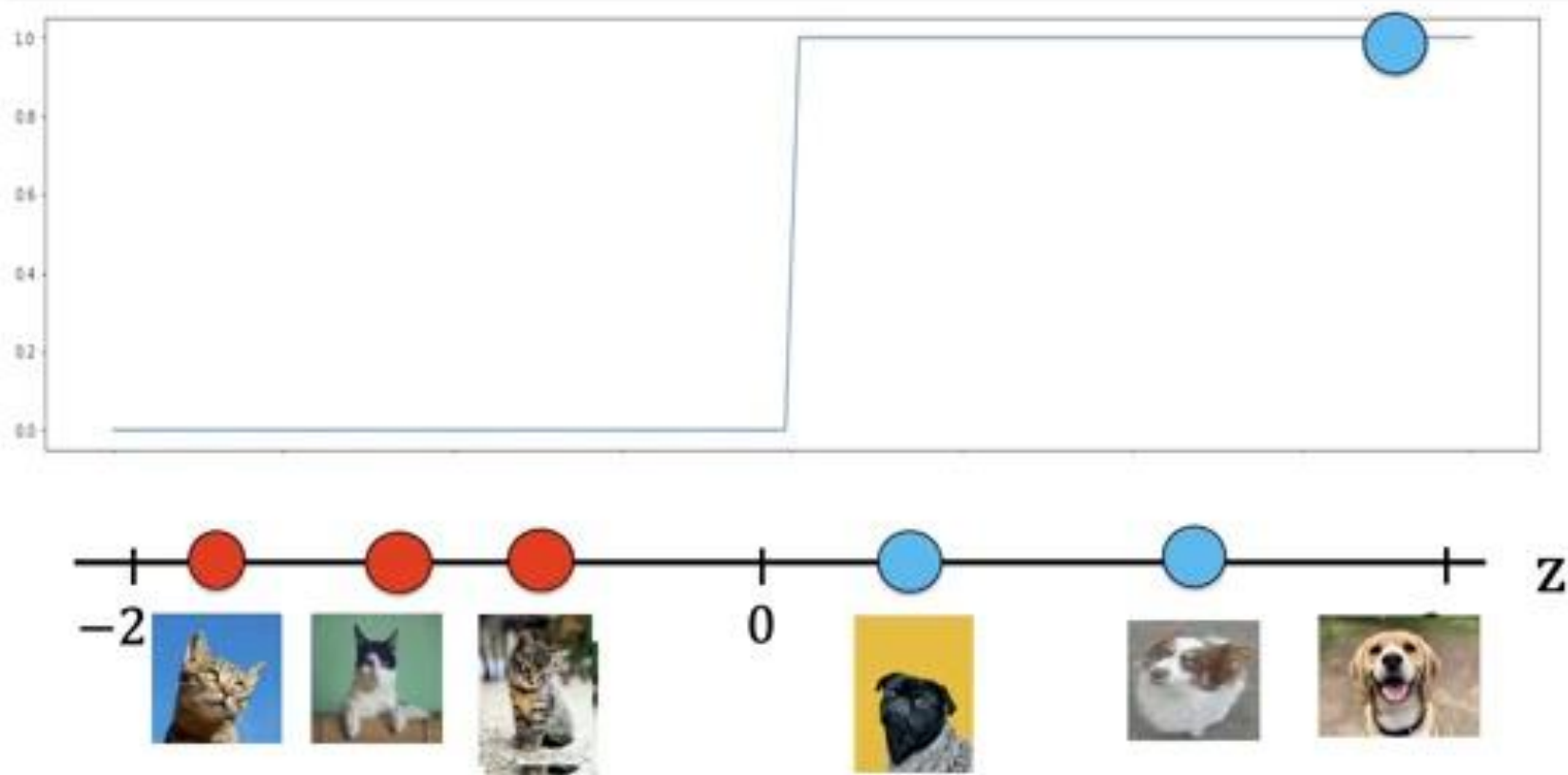
$$\hat{y} = f(z)$$



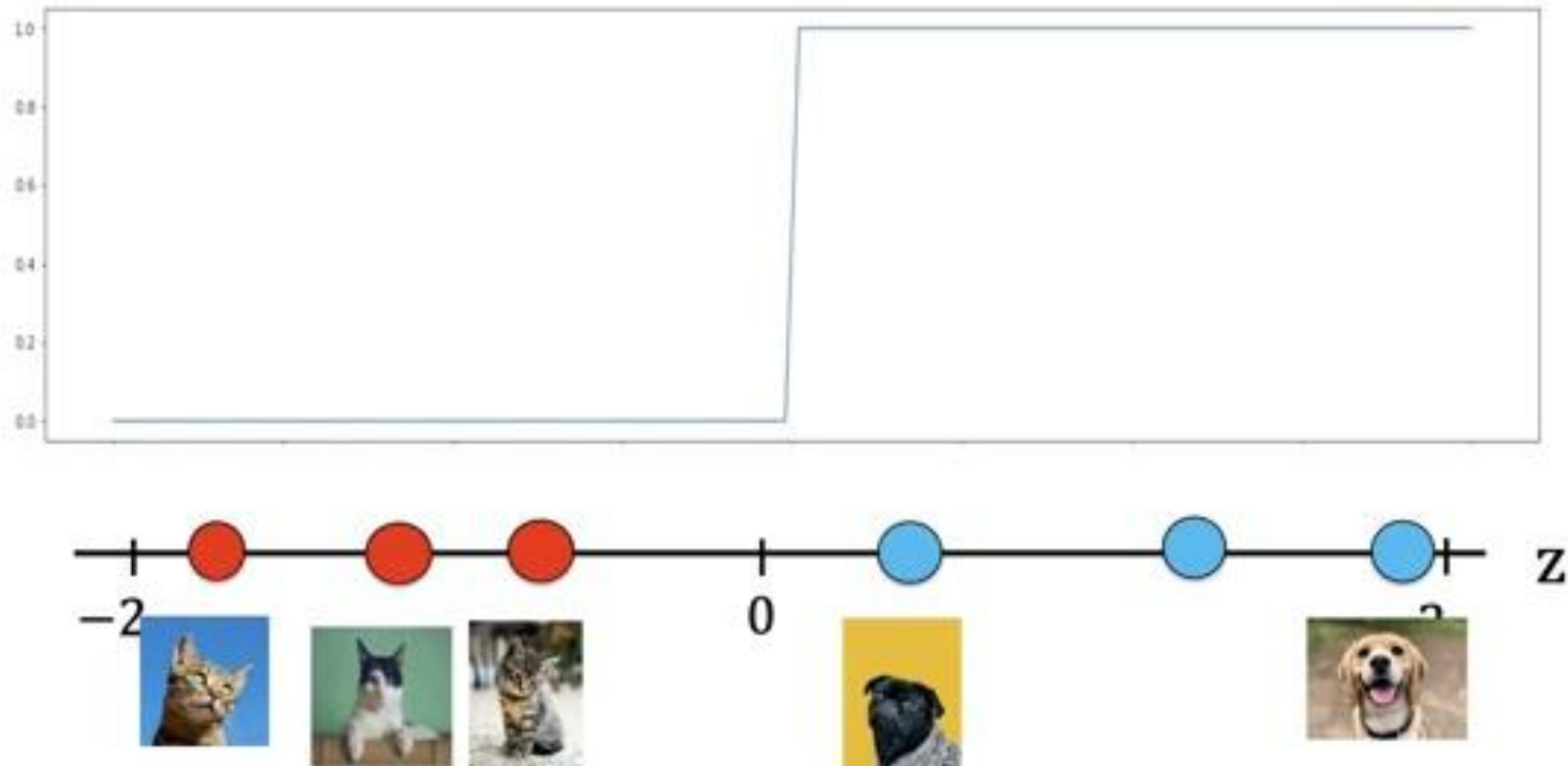
$$\hat{y}_1 = f(\text{Image of a dog})$$



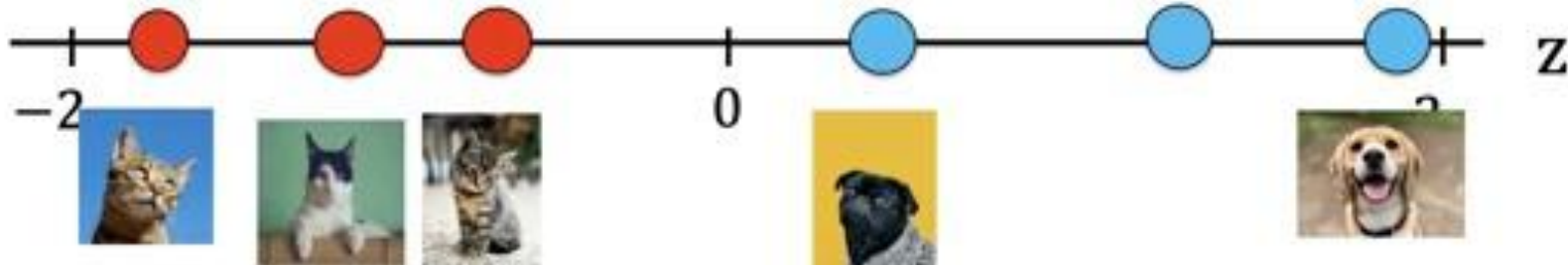
$$\hat{y}_1 = f(\text{Image of a dog}) = 1$$



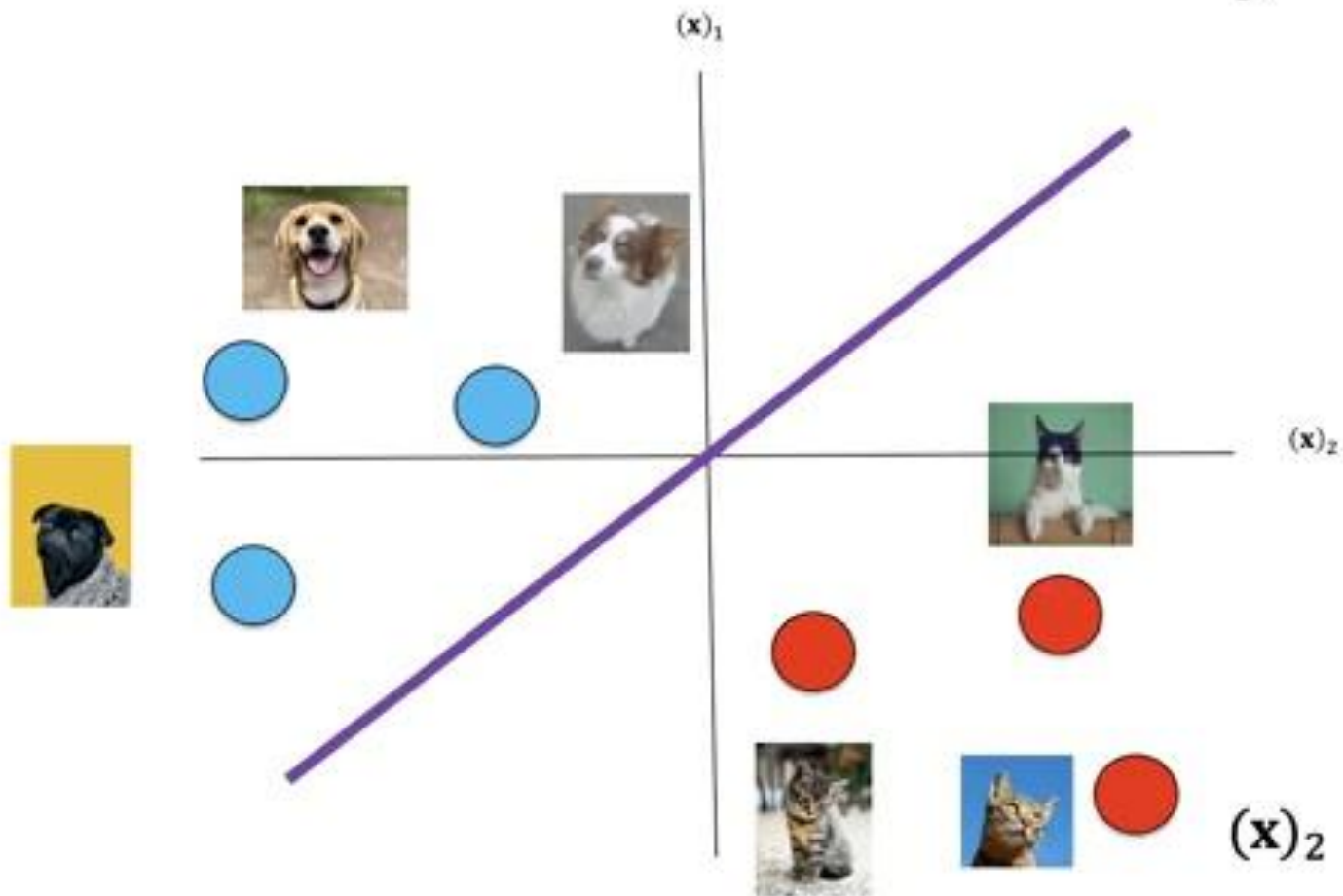
$$\hat{y}_1 = f(\text{cat image})$$

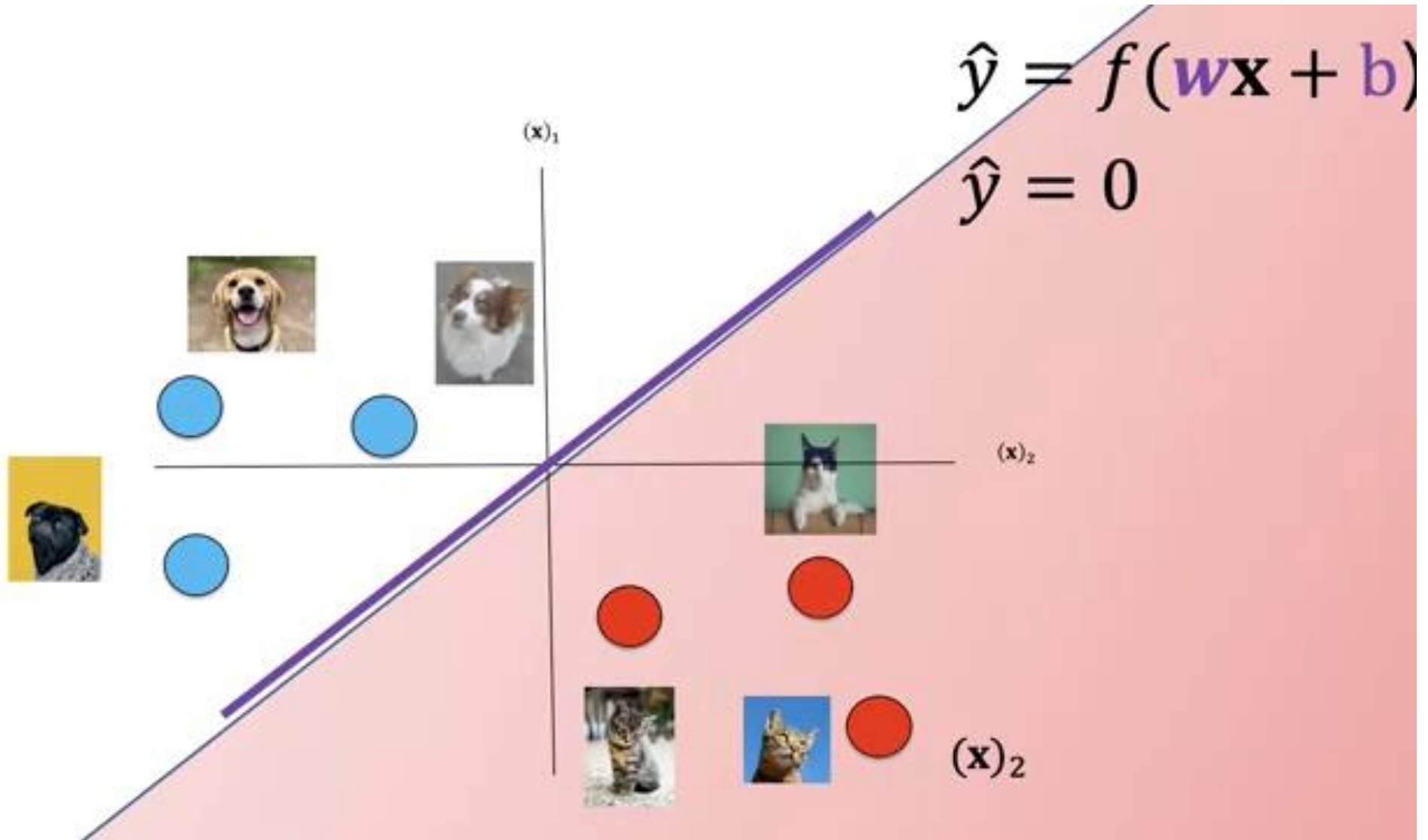


$$\hat{y}_1 = f(\text{cat image}) = 0$$

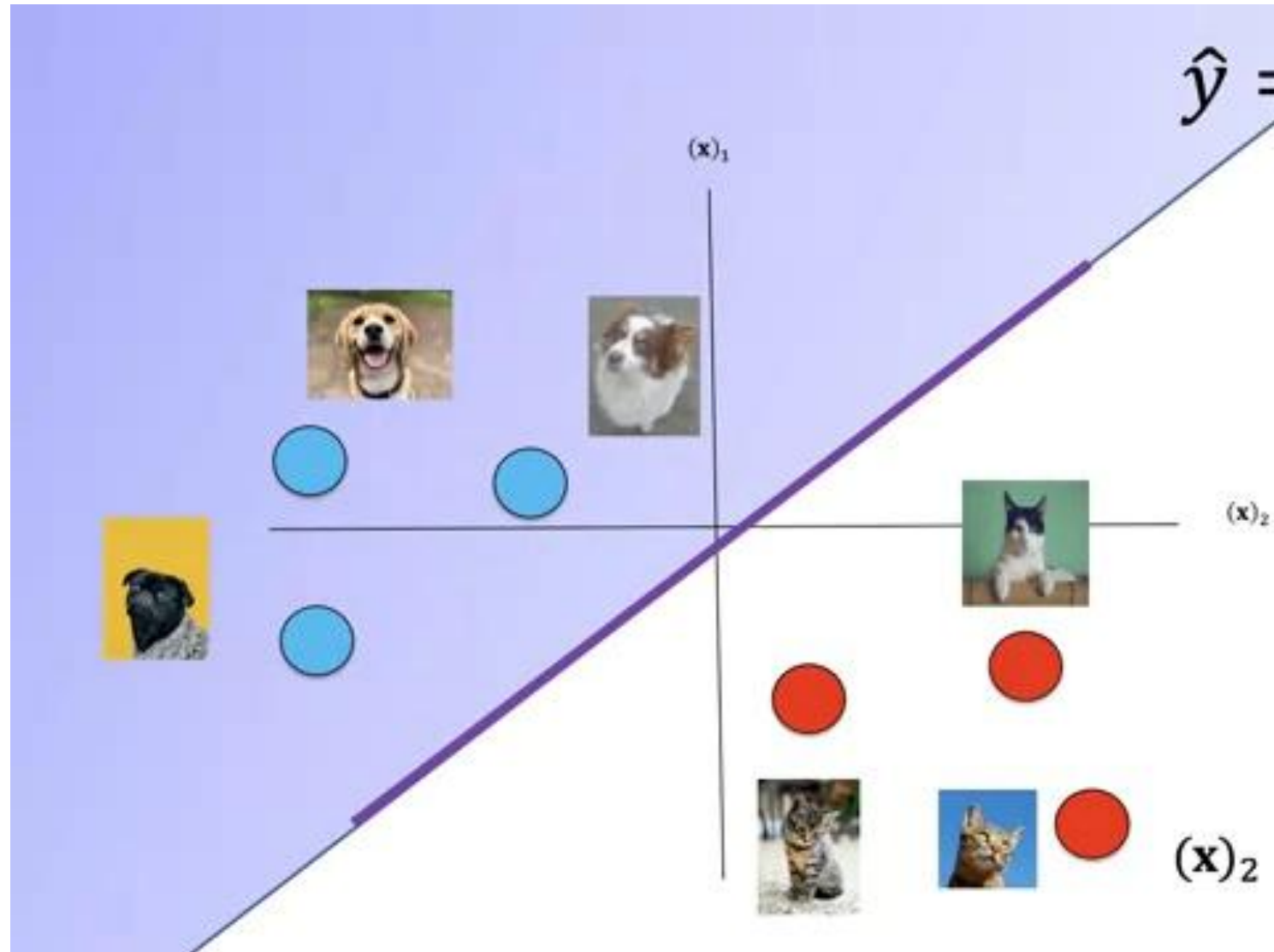


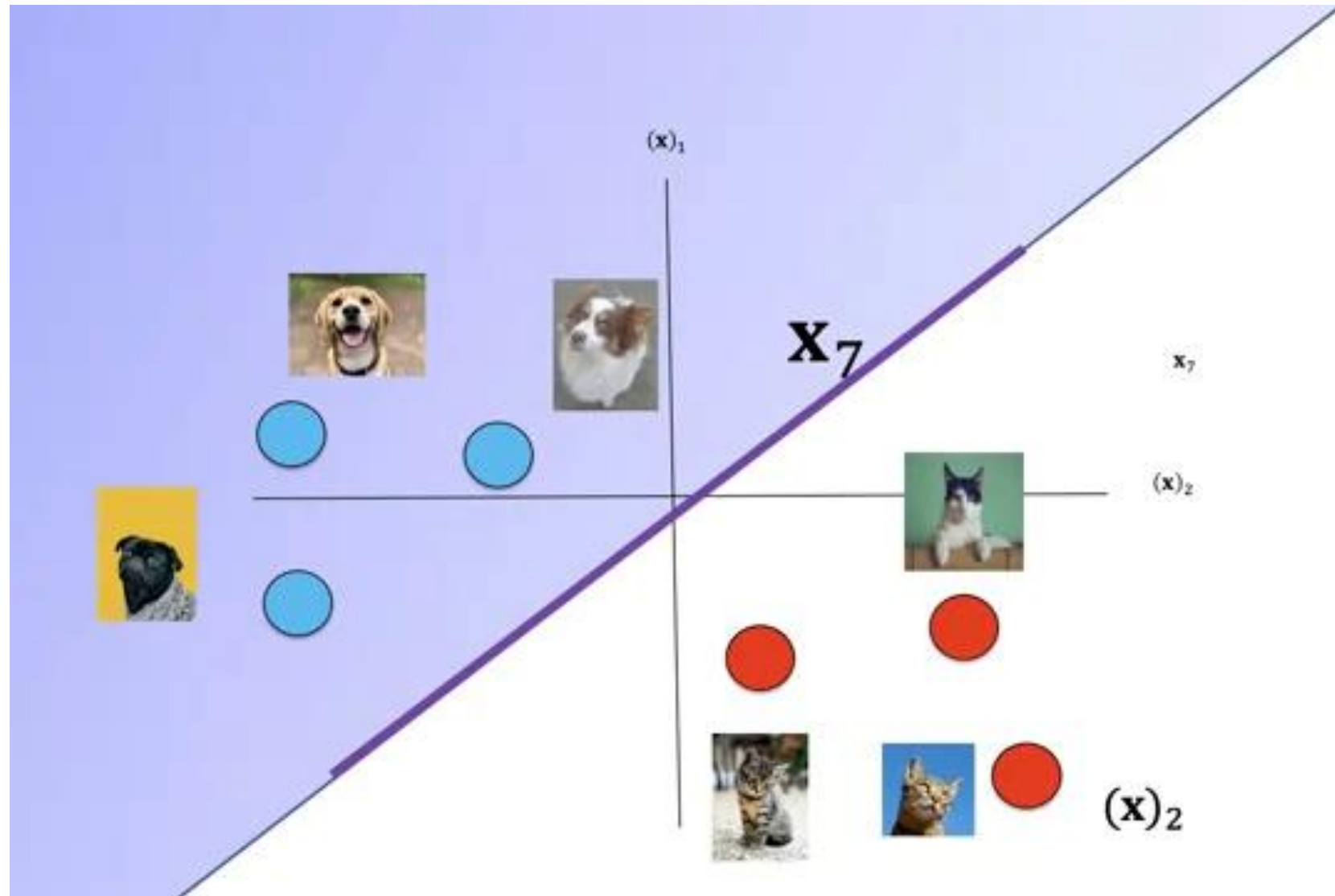
$$\hat{y} = f(\mathbf{w}\mathbf{x} + \mathbf{b})$$

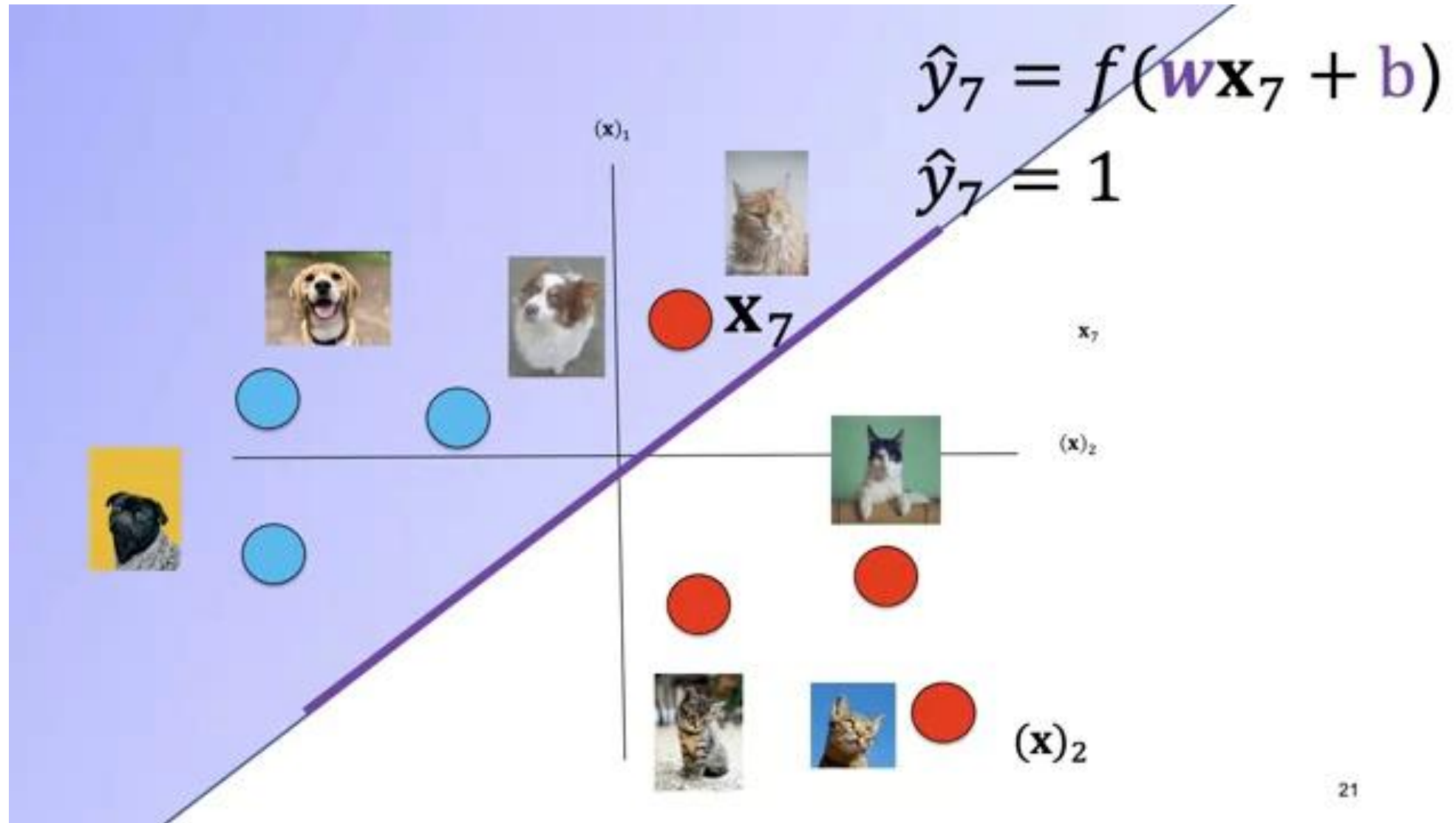




$$\hat{y} = f(\mathbf{w}\mathbf{x} + \mathbf{b})$$

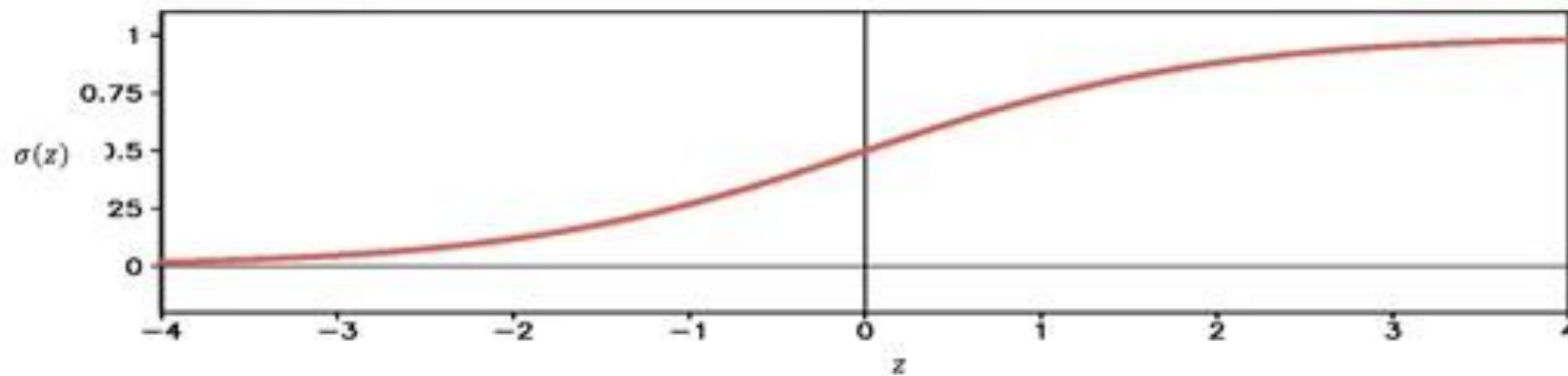






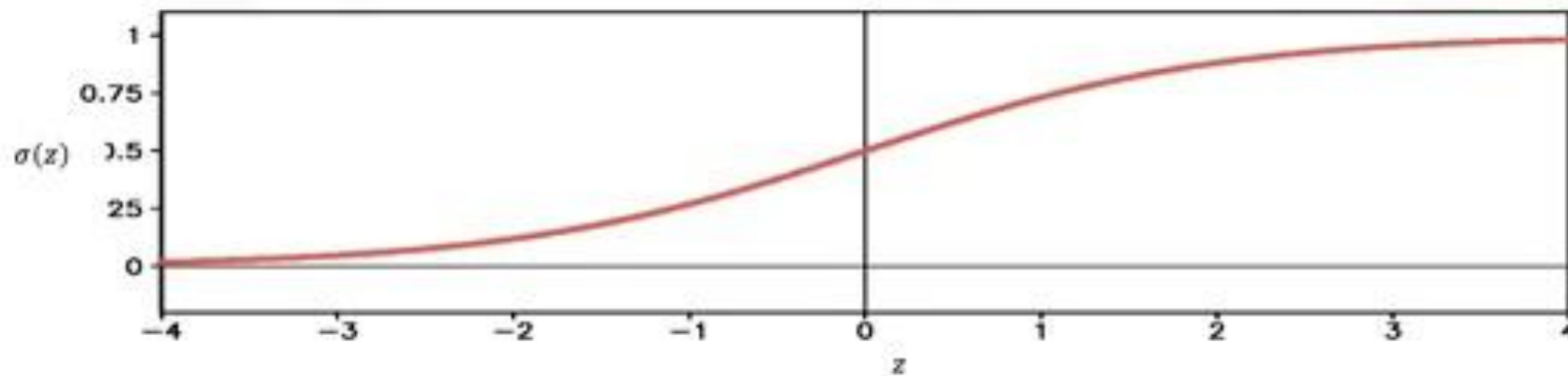
Logistic Regression

- Logistic Function



Logistic Regression

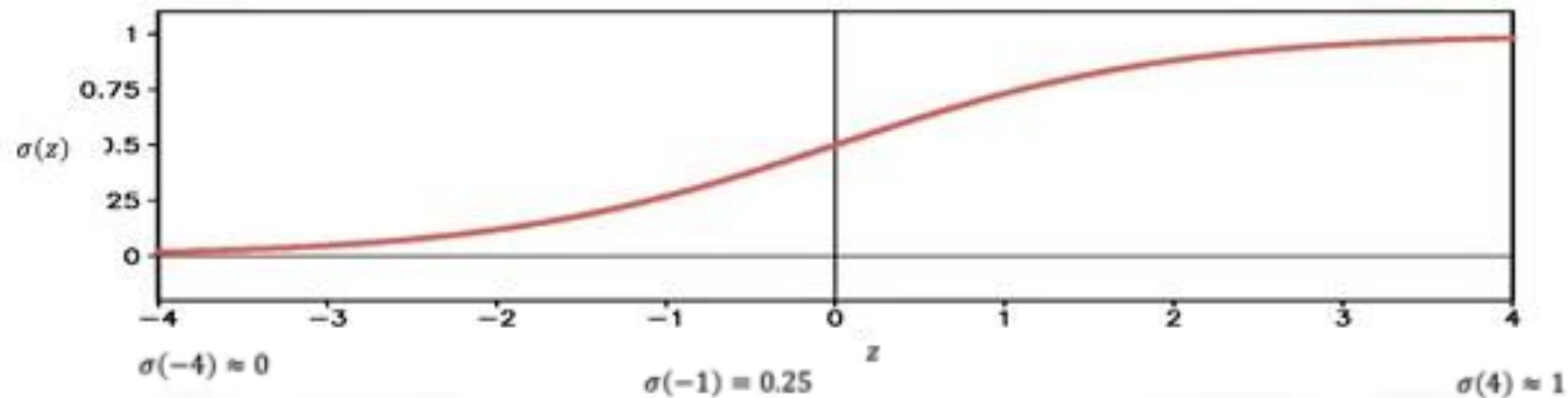
- Logistic Function



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

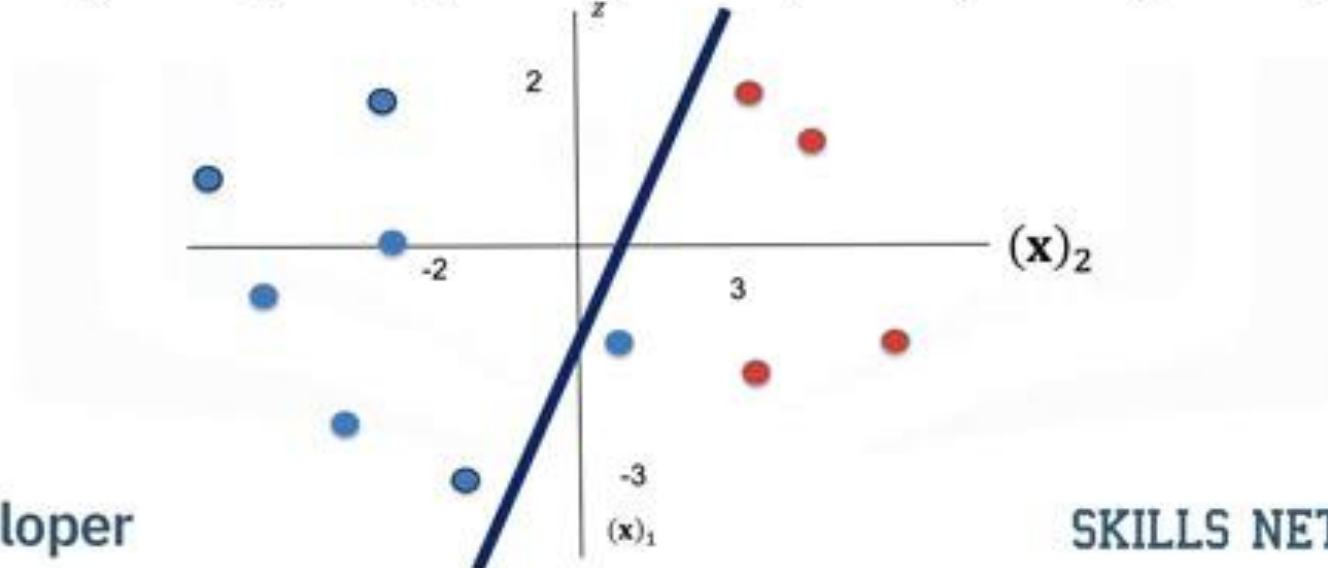
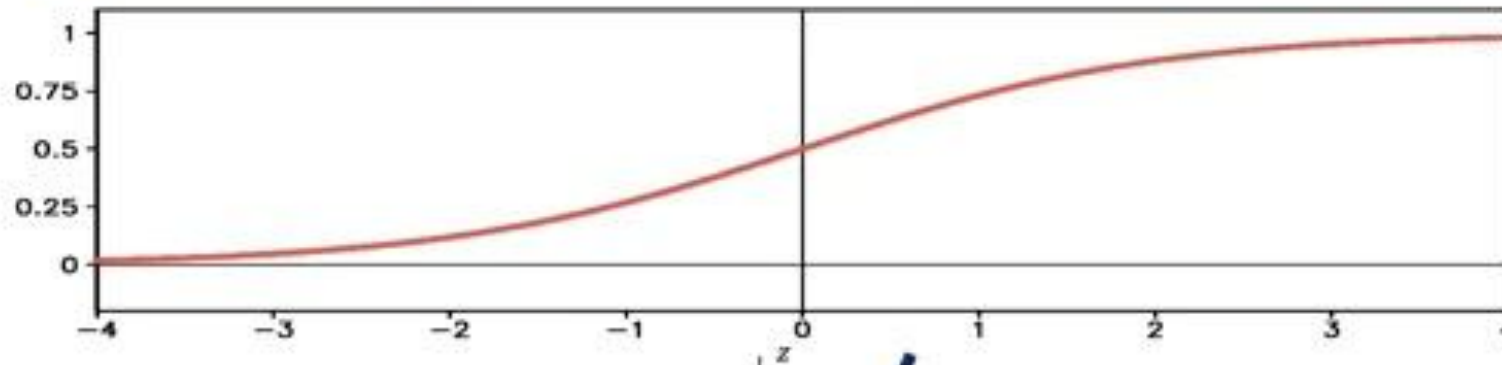
Logistic Regression

- Logistic Function

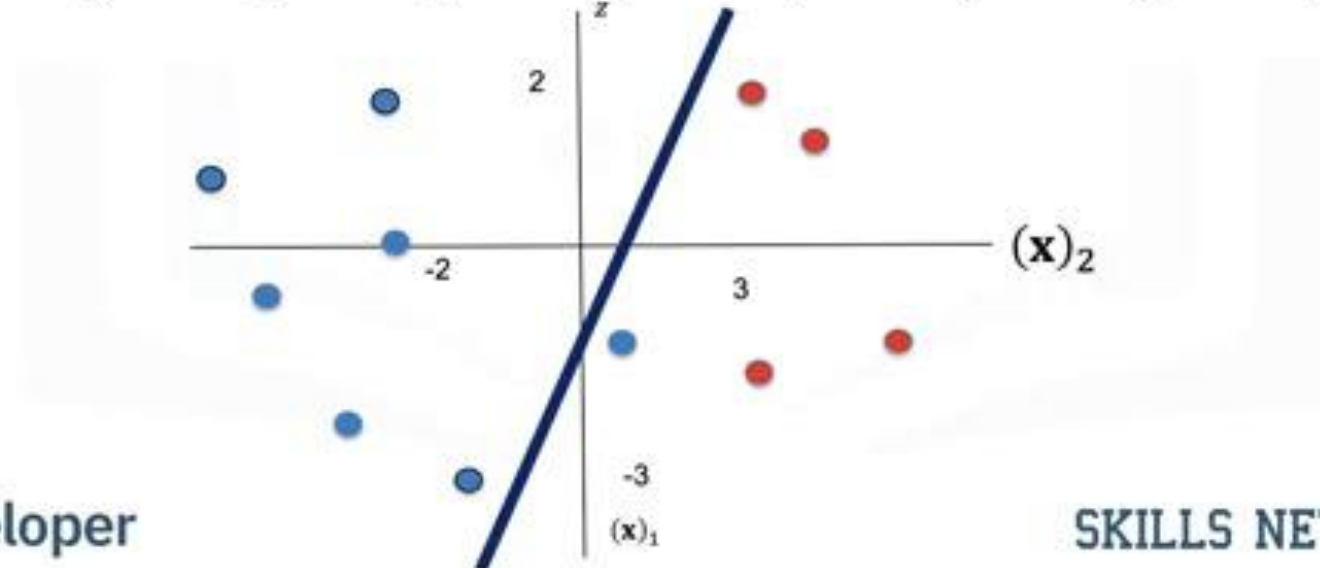
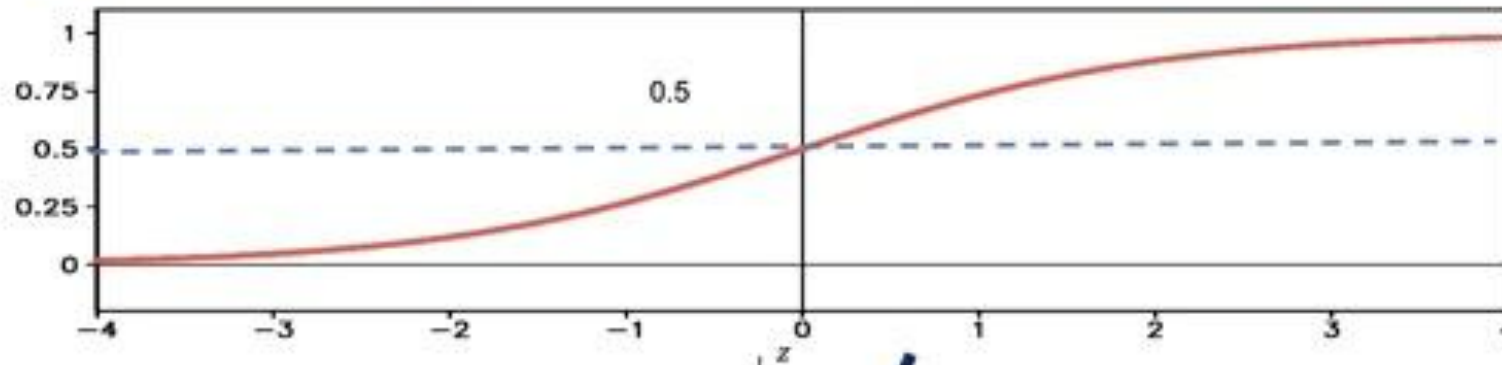


$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

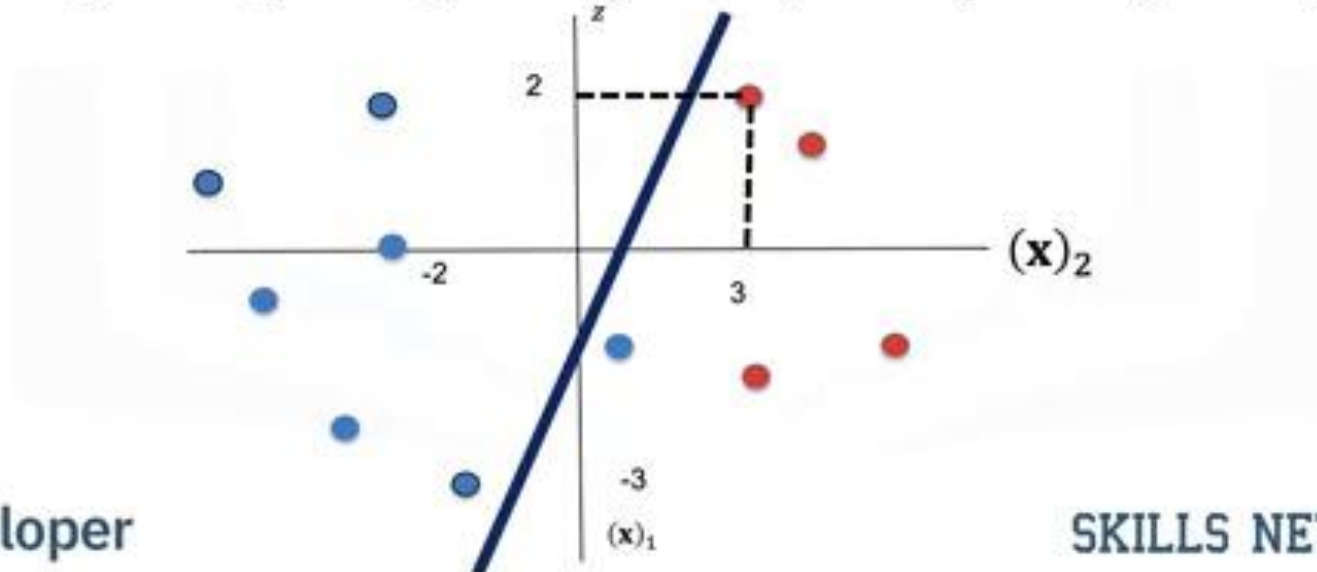
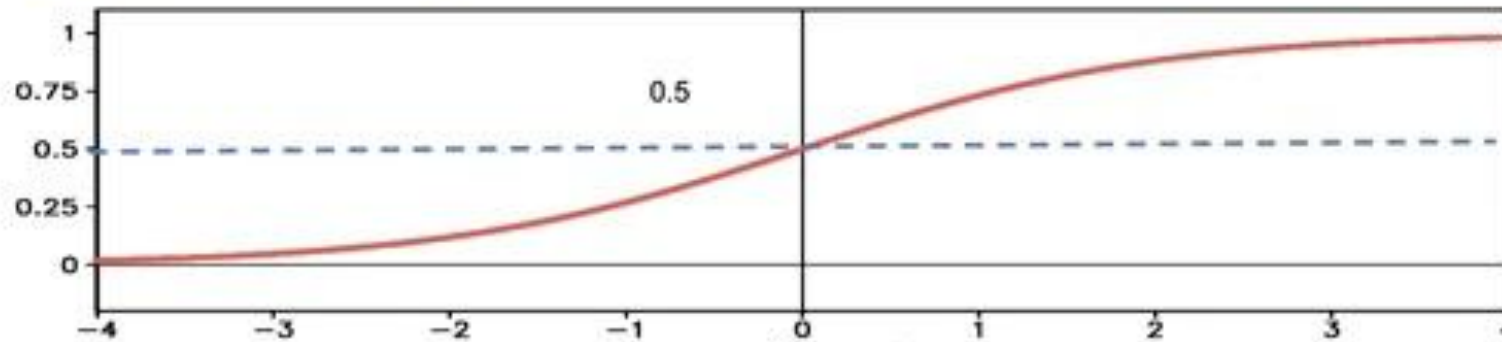
Logistic Regression



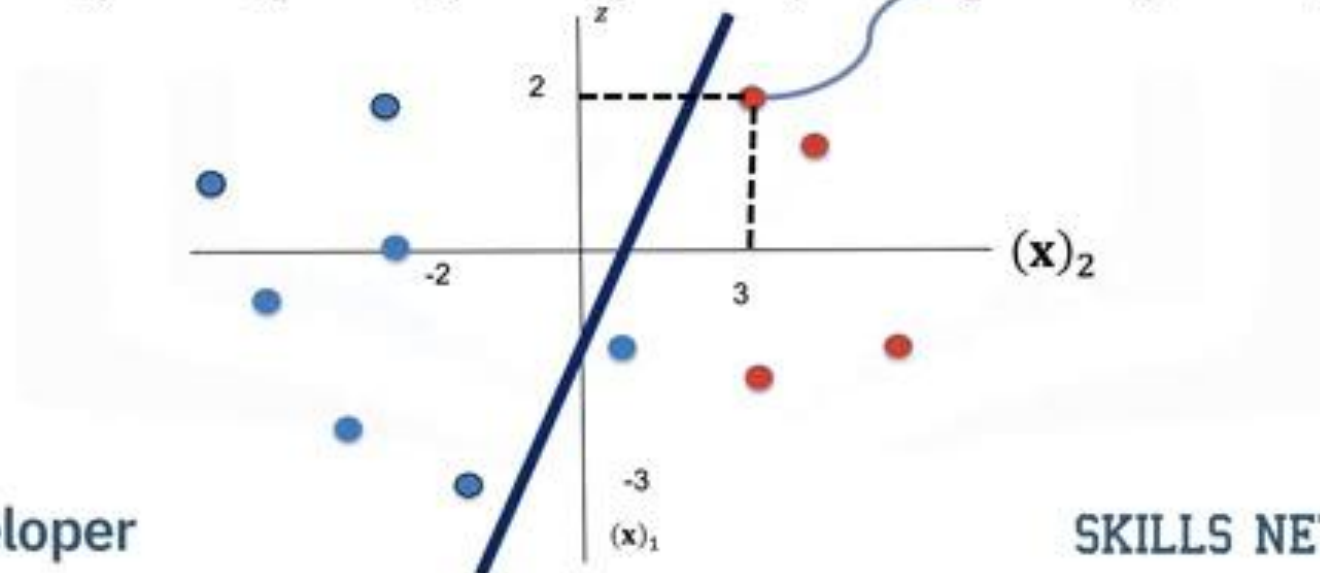
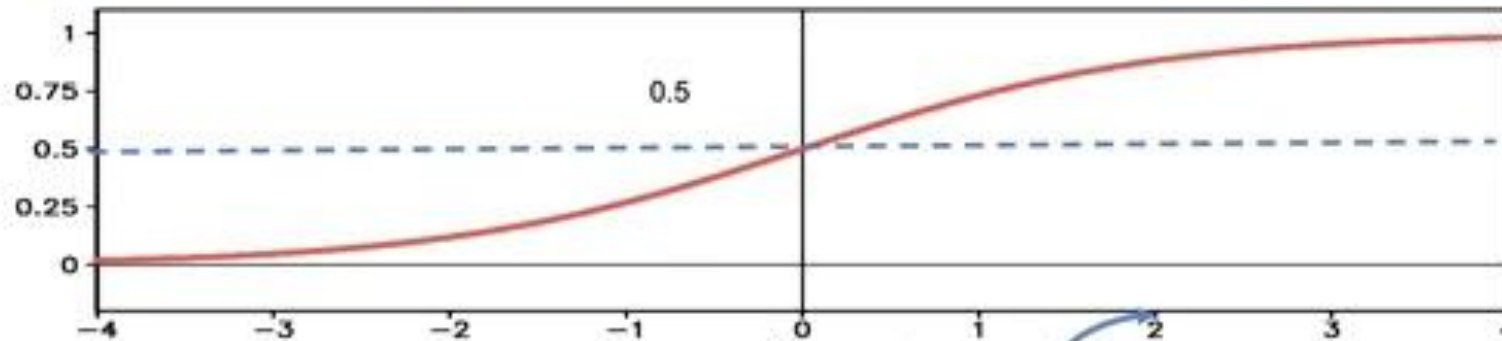
Logistic Regression



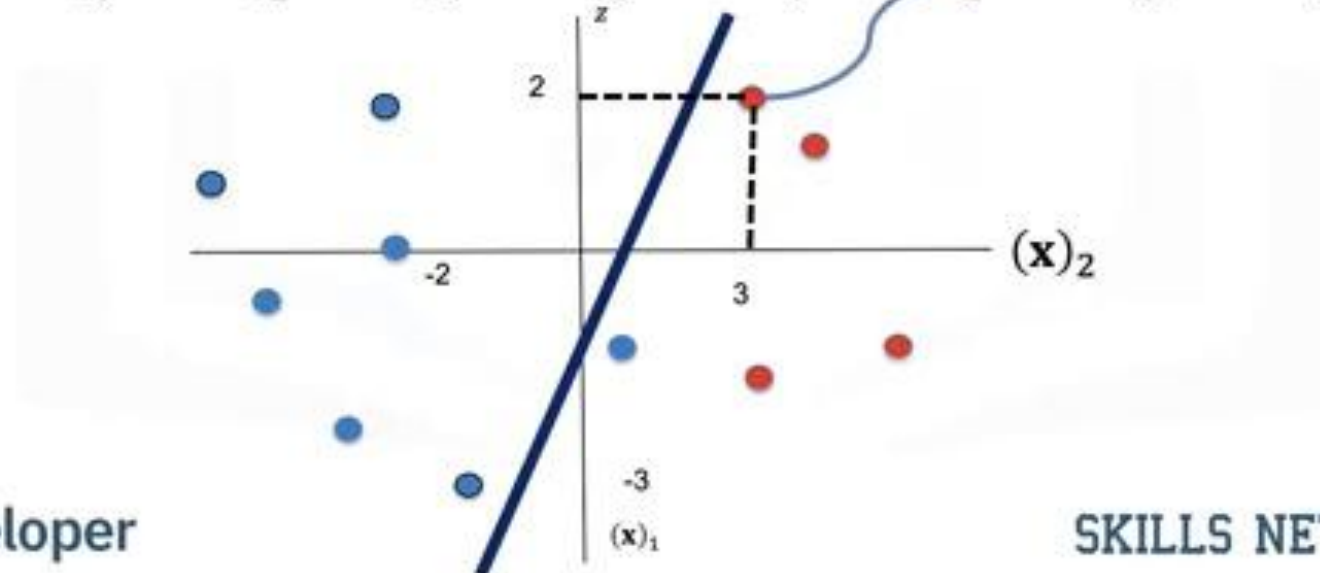
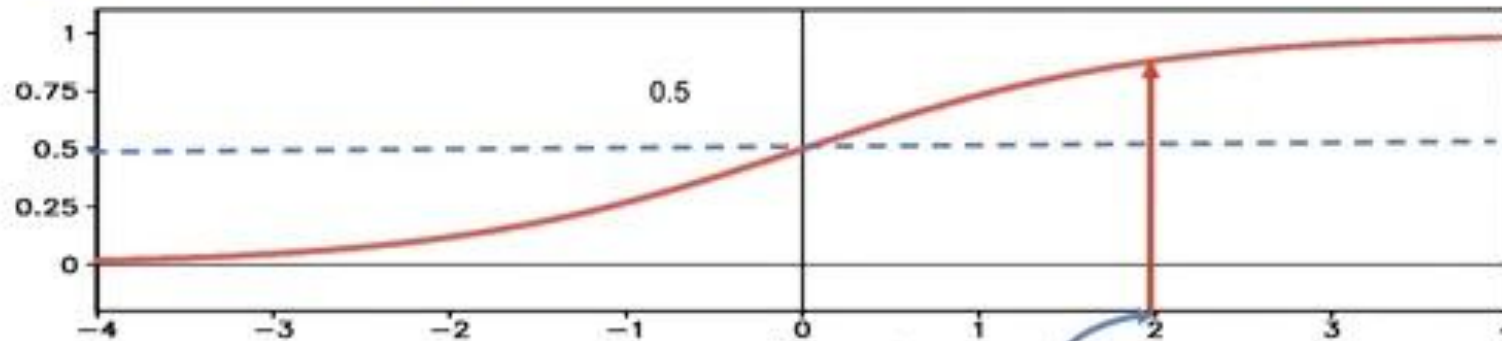
Logistic Regression



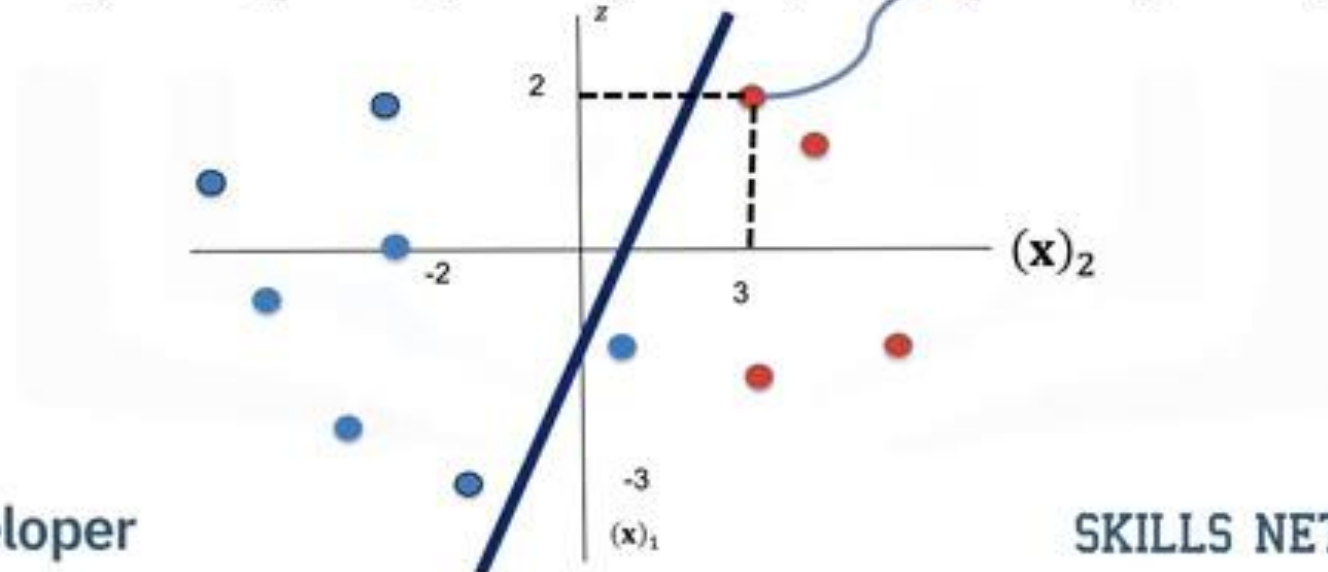
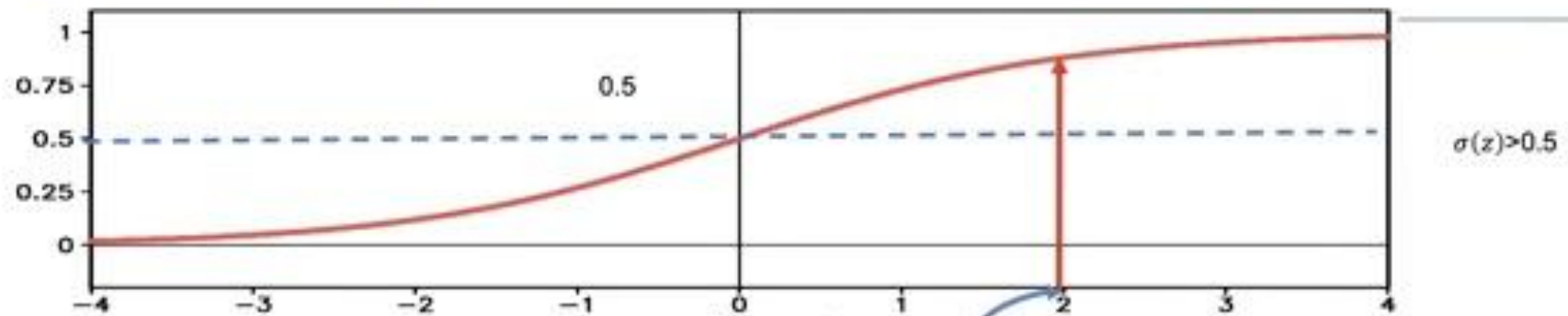
Logistic Regression



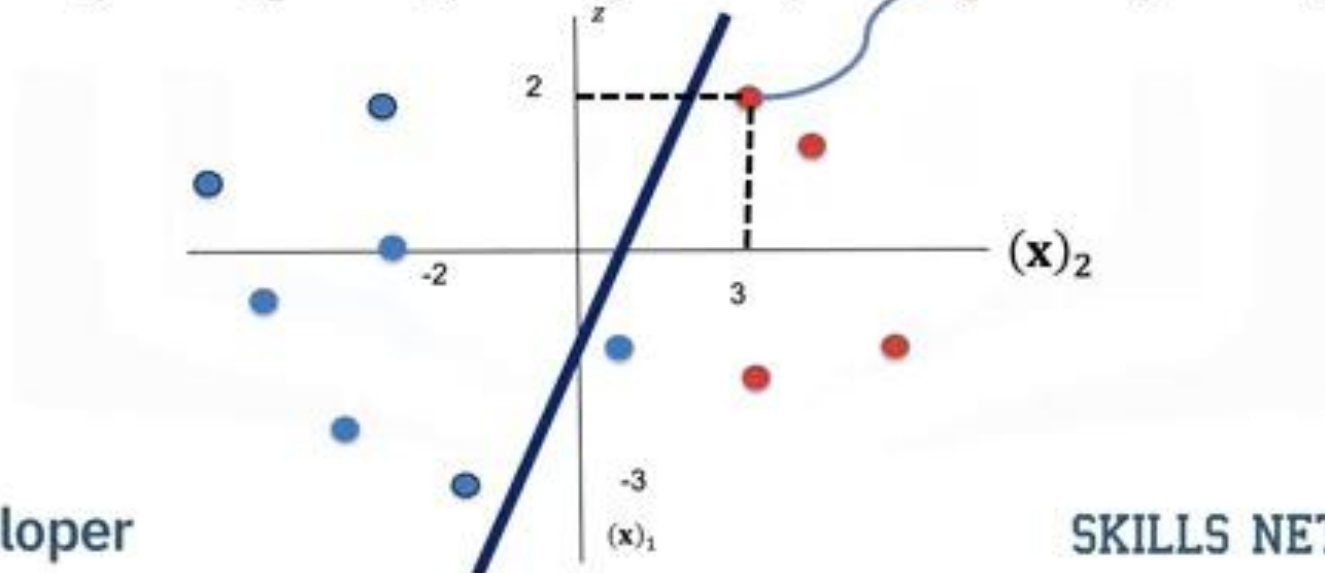
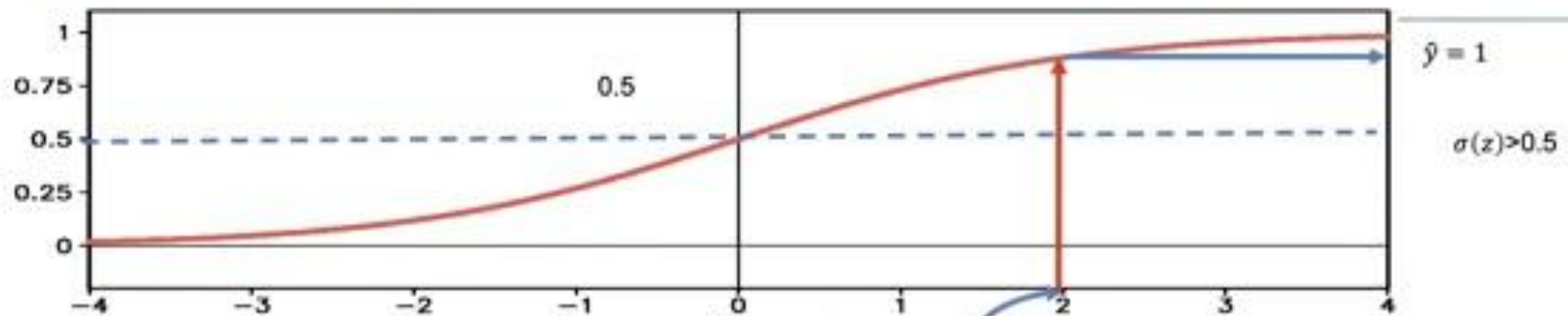
Logistic Regression



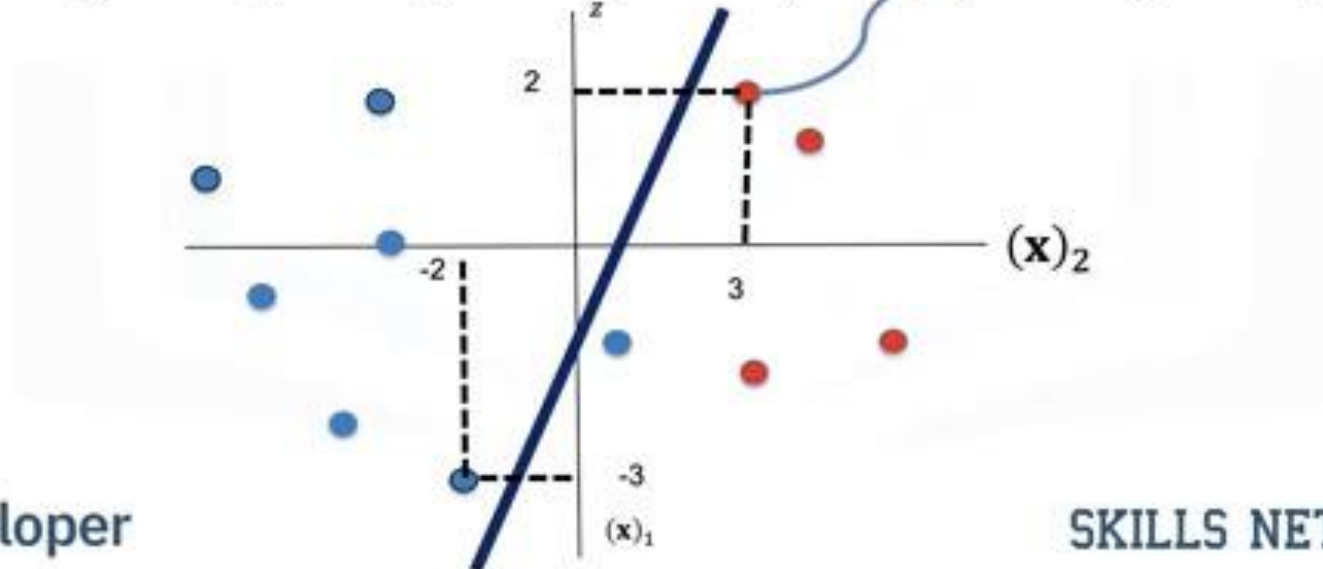
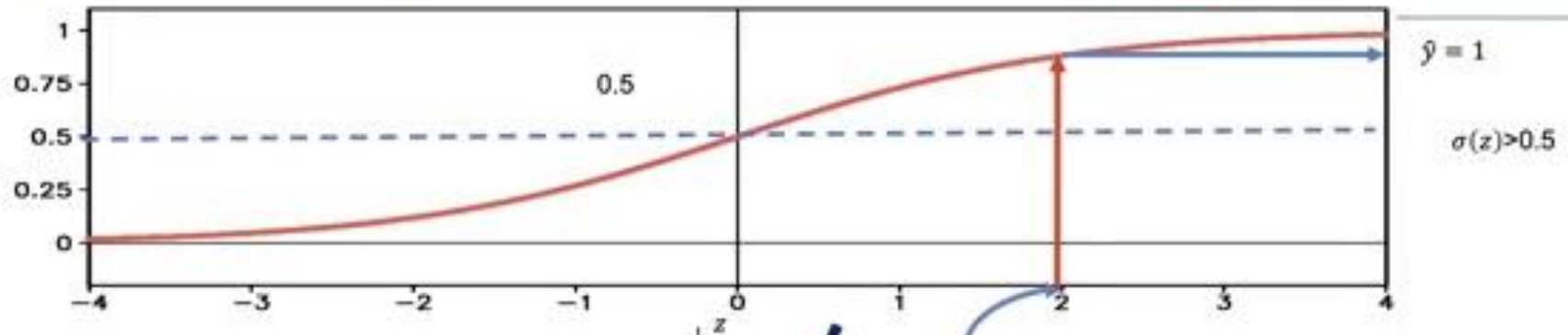
Logistic Regression



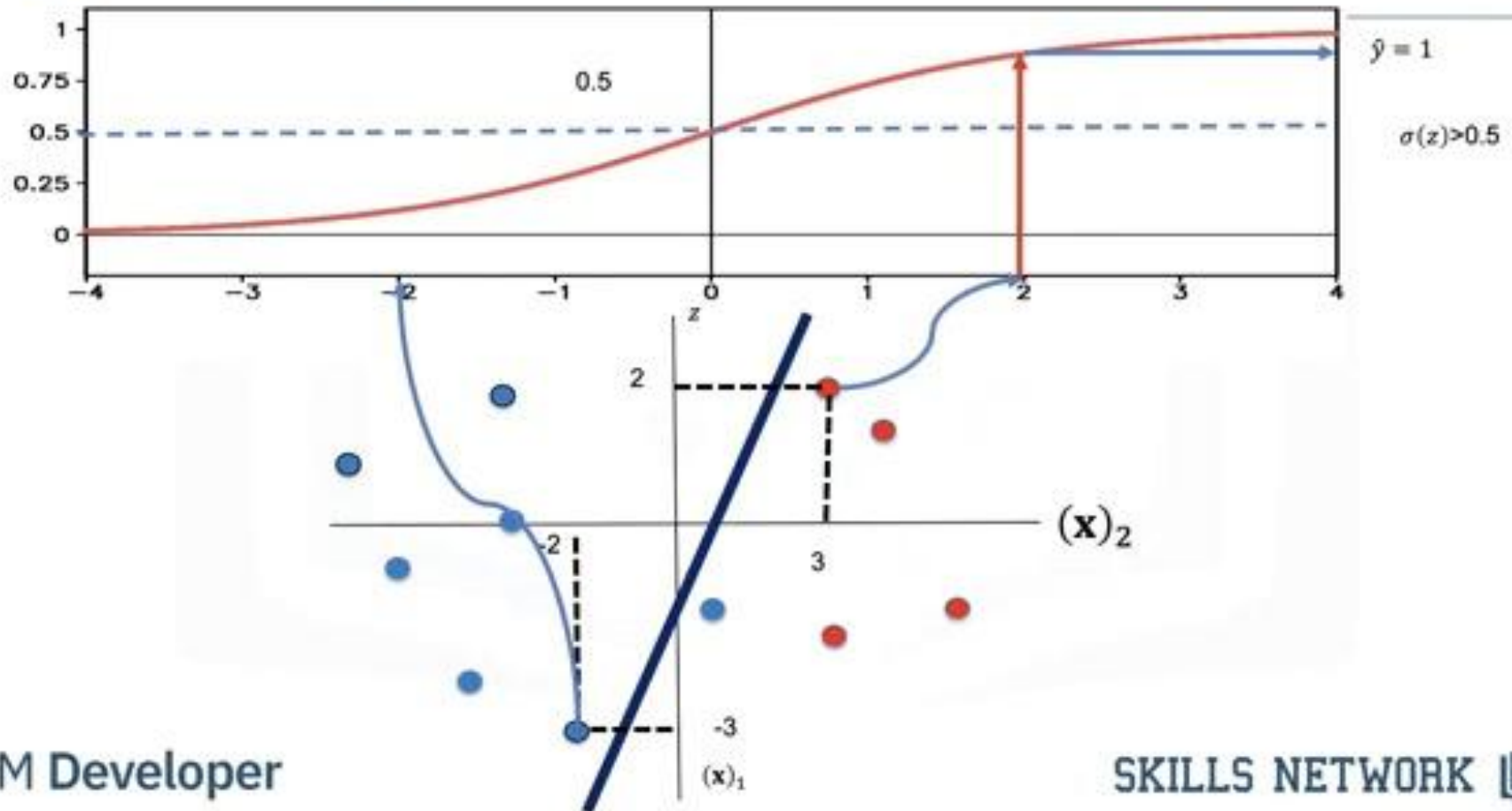
Logistic Regression



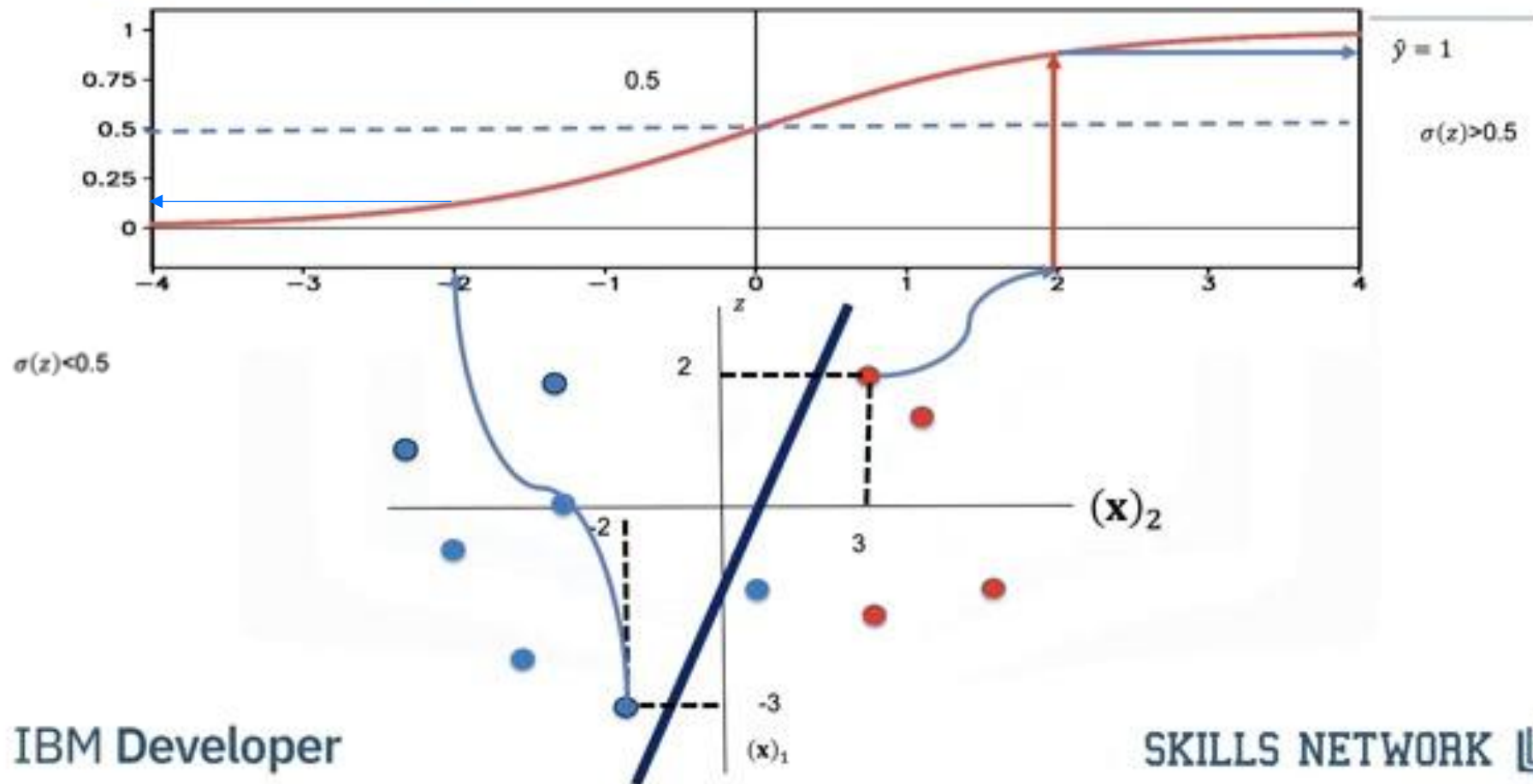
Logistic Regression



Logistic Regression



Logistic Regression



$$\overset{\sigma}{p}(\text{Image of a Dog}) = 0.92 \text{ Probability this image is a Dog}$$

$$\overset{\sigma}{p}($$

 $)$

= 0.92 Probability this image is a Dog

$$1 - p($$

 $)$

= 0.08 Probability this image is a Cat

$p(\text{cat})$



$1 - p(\text{cat})$



$$p(\text{) = 0.10$$

Probability this image is a Dog

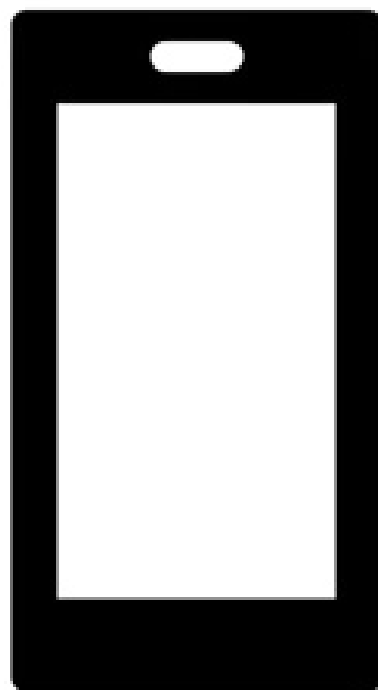
$$1 - p(\text{)$$

$$p(\text{) = 0.10$$

Probability this image is a Dog

$$1 - p(\text{) = 0.9$$

Probability this image is a Cat





$$\hat{y} = f(\text{cat image})$$

$$\hat{y} = 0$$



-
- https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Examples

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.linear_model import LogisticRegression
>>> X, y = load_iris(return_X_y=True)
>>> clf = LogisticRegression(random_state=0).fit(X, y)
>>> clf.predict(X[:2, :])
array([0, 0])
>>> clf.predict_proba(X[:2, :])
array([[9.8...e-01, 1.8...e-02, 1.4...e-08],
       [9.7...e-01, 2.8...e-02, ...e-08]])
>>> clf.score(X, y)
0.97...
```